Java におけるオブジェクト指向

ver. 1.0

presented by t.ogisawa®

目次

1. オブジェクト指向の基礎 5
1.1. クラス（class） 5
1.1.1. クラスの定義 5
1.1.2. フィールド 5
1.1.3. メソッド 6
1.1.4. インスタンス化（new） 7
1.2. カプセル化 9
1.2.1. ゲッターとセッター（getter／setter） 9
1.2.2. インナークラス（Inner Class） 10
1.3. コンストラクタとガベージコレクション 12
2. 継承（extends） 15
2.1. スーパークラスとサブクラス 15
2.2. 自分自身（this） 15
2.3. スーパークラスの参照（super） 16
2.3.1. 参照変数とキャスト 16
2.4. Object クラス 17
2.5. クラスに属しているか調べる（instanceof） 17
3. ポリモーフィズム（多態性） 19
3.1. オーバロード 19
3.2. オーバライド 19
3.3. 抽象化 21
3.4. 抽象クラス 21
3.5. インタフェース（interface） 22
3.5.1. インタフェースの実装（implements） 22
3.5.2. インタフェース修飾子（public, ...） 23
3.5.3. インタフェースの継承（extends） 23
3.5.4. default と static メソッド [Java8～]、private メソッド[Java9～] 23
4. パッケージとアクセス修飾子 27
4.1. インポート（import） 27
4.2. パッケージ文（package） 28
4.3. アクセス修飾子 29
4.3.1. クラスの修飾子 29
4.3.2. フィールドの修飾子 29
4.3.3. メソッドの修飾子 29
4.3.4. コンストラクタの修飾子 29
5. 変数とその扱い 31
5.1. 基本型とラッパークラス 31
5.2. 数値演算（Math クラス、Random クラス） 31
5.3. 日付／時刻（Date-Time API）[Java8～] 33
5.4. 文字列 34
5.4.1. 文字列（String） 34
5.4.2. 文字列長を得る（length, isEmpty） 34
5.4.3. 等しいかどうか調べる（equals, equalsIgnoreCase） 34
5.4.4. 文字列を整形する（format） 35
5.4.5. その他メソッド 36
5.4.6. StringBuffer と StringBuilder 38

5.5. 配列(array) 39
5.5.1. 配列の作成 39
5.5.2. java.util.Arrays クラス 40
5.6. 列挙型(Enum 型) 41
6. コレクション(集合) 43
6.1. コレクションフレームワーク 43
6.2. Collection インタフェースでの iterator 活用 44
6.3. リスト(List) 44
6.3.1. ArrayList 44
6.3.2. LinkedList 45
6.4. セット (Set) 45
6.4.1. HashSet 45
6.4.2. TreeSet 46
6.5. マップ(Map) 46
6.5.1. HashMap 46
6.5.2. TreeMap 47
6.6. コレクションのマルチスレッド対応 48
6.7. ジェネリクス 48
6.7.1. 境界ワイルドカード型ジェネリクス 49
6.7.2. ジェネリクスタイプ 49
6.8. 変更不可能なリスト、セットおよびマップの作成 [Java9～] 50
7. 関数型インタフェースとラムダ式 52
7.1. 無名クラス 52
7.2. ラムダ式[Java8～] 52
7.3. メソッド参照 53
7.4. 関数型インターフェース 53
8. ストリーム API(Stream API)[Java8～] 56
8.1. ストリーム生成 56
8.2. ストリーム中間処理 57
8.3. ストリーム終端処理 59
8.3.1. collect メソッド 59
8.3.2. reduce メソッド 60
8.3.3. その他の終端処理メソッド 61
9. 例外処理 63
9.1. 例外とは 63
9.2. 例外をキャッチする(try, catch, finally) 63
9.3. try-with-resources 文 [Java7～, 9～] 64
9.4. 例外を投げる(throw, throws) 65
9.5. 例外オブジェクト(Exception) 66
9.6. Optional クラス [Java8～] 67
10. 入出力(File-I/O) 70
10.1. ファイル操作 70
10.2. バイナリファイルを読み書きする(FileInputStream／FileOutputStream) 71
10.3. テキストファイルを読み書きする(InputStreamReader／OutputStreamWriter) 72
10.4. File I/O で Stream を扱う手順 73
11. アノテーション 75
11.1. 標準アノテーション 75
11.2. アノテーションのカスタマイズ 76

本テキストは、現在多くの本番環境で利用されているバージョン Java８（JDK 8）と一部 Java９（JDK9）を元に解説しています。（尚、Java8, 9 の機能については各節タイトルに[Java8～]または [Java9～]と記載されています。）

1. オブジェクト指向の基礎

1. オブジェクト指向の基礎

オブジェクト指向言語では、プログラムの単位がオブジェクトと呼ばれるモジュールが、相互に協調し合いながら処理を行います。

1.1. クラス（class）

クラス（class）はオブジェクト指向プログラミングの基本的な仕組みを提供するもので、クラスを元に生成されたオブジェクトがどう動くのかその機能と動作を決める雛形です。クラスにより生成されたオブジェクトをインスタンスと呼びます。

1.1.1. クラスの定義

クラスには、インスタンスが所有するフィールド（属性、アトリビュート、プロパティ とも呼ばれます）や、メソッド（関数、ファンクションと呼ばれることもあります）などを定義します。
インスタンスは自分のクラス定義に従い生成され何らかの処理を行い必要がなくなったら消滅します。
オブジェクト指向では、こうして生成されたある役割を持った独立したオブジェクトが、時として子となるオブジェクトや他のオブジェクトと協調しながら処理を実行していきます。

```
class クラス名 {
   型 フィールド;
      ...
   型 メソッド() { ... }
      ...
}
```

1.1.2. フィールド

クラス定義のブロック直下に宣言された変数のことを、インスタンスフィールド（インスタンス変数）と呼びます。

```
class クラス名 {
   型 変数名;
}
```

例えば、int 型の変数 a を定義するには次のようにします。

```
class クラス名 {
   int a;
}
```

フィールド修飾子（public, ...）

変数の修飾子には、public、protected、private、static、final を指定することができます。

```
class クラス名 {
   修飾子 int testVariable;
   ...
}
```

クラスフィールド（static）

```
class AccessCounter {
   static int totalCount = 0;
   int count;
}
```

フィールドに static キーワードが追加された場合、クラス定義ですぐに使えるクラスフィールドと呼ばれます。個別のインスタンス毎に存在せずクラスに紐づく唯一の変数であり、例えば全インスタンス共通の値を設定・取得できるフィールドなどに使用できます。クラスフィールドは、”クラス名.フィールド名”でアクセスできます。

```
System.out.println(AccessCounter.totalCount);
```

定数（static final）

static フィールドにさらに final キーワードを使いすることで、変更不可能な定数を表現することが出来ます。

定数は通常大文字で記述するのが習慣です。

```
class Math {
   public static final double PI = 3.14159265358979323846;
}
```

1.1.3. メソッド

クラス内部で定義される機能をメソッドと呼びます。このメソッドをインスタンスからコールするインスタンスメソッドと呼ばれます。

```
class クラス名 {
   型 メソッド名(引数) {
      ...
   }
}
```

メソッドの例を下記に示します。

```
class MethodTest {
   public static void main(String[] args) {
      System.out.println(add(3, 5));
   }
   static int add(int x, int y) {
      return (x + y);
   }
}
```

メソッドにはいくつかの 引数 を指定することができます。引数には、型と引数名のペアを 0 個以上指定することができます。

```
class クラス名 {
   型 メソッド名(型 1 引数名 1, 型 2 引数名 2) {
      ...
   }
```

```
}
```

メソッドの 戻り値 は return を用いて呼び出し元に返します。

```
class クラス名 {
   型 メソッド名(引数) {
      ...
      return(値)
   }
}
```

メソッド修飾子には、public、protected、private、static、final、 abstract、synchronized を指定することができます。abstract を含む場合は抽象メソッドで、メソッドの宣言だけで実体が無いなので、処理の中身は書きません。

```
class クラス名 {
   修飾子 void testFunctionA(int a) {
      ...
   }
   修飾子 void testFunctionB(int a);
}
```

static キーワードの追加された場合、インスタンスにしなくてもクラス定義から使えるものでクラスメソッドと呼ばれます。”クラス名.メソッド名”でアクセスされます。

1.1.4. インスタンス化（new）

あるクラスのインスタンスを作成するには、new を用います。例えば、TestClass という名前のクラスのインスタンスを作るには次のようにして、クラスを new して作成したインスタンスを同じクラス型の変数に代入します。
この変数のことを参照変数と呼びます。

```
TestClass testClass = new TestClass();
```

クラスとその使用例
例えば下記の例では、Person という名前のクラスを定義しています。Person クラスは、name と age というフィールドを持っています。introduceString というメソッドを持っています。

```
public class Person {
   public String name;
   public int age;

   public String introduceString(){
      return “My name is ”+name+”, My age is “+age+” years old.”;
   }
}
```

ここで作成した Person クラスを使用するには次のようにします。まず、new でインスタンス を作成して、同じ Person 型の参照変数 abe や yamada にそのインスタンスの参照を入れます。

フィールドや、メソッドは、その参照変数を使ってアクセスされます。

・インスタンスフィールドへの参照：”参照変数名.フィールド名”
・インスタンスメソッドの呼び出し：”参照変数名.メソッド名”

```
class PersonTest {
  public static void main(String[] args) {
    Person abe = new Person(); // 阿部さんオブジェクトを作る
    abe.name = "Abe"; // 阿部さんの名前を設定する
    abe.age = 26;// 阿部さんの年齢を設定する

    Person yamada = new Person(); // 山田さんオブジェクトを作る
    yamada.name = "Yamada";  // 山田さんの名前を設定する
    yamada.age = 32;   // 山田さんの年齢を設定する

    System.out.println(abe.name);
    System.out.println(abe.age);
    System.out.println(abe.introduceString());
    System.out.println(yamada.name);
    System.out.println(yamada.age);
    System.out.println(yamada.introduceString());
  }
}
```

1.2. カプセル化

1.2.1. ゲッターとセッター（getter／setter）

カプセル化とは、データ（フィールド）と、そのデータを操作するメソッドをひとまとめにし（カプセルに入れ）、外部からデータを直接（不正）に書き換えられないように保護します。データの管理責任と関連する機能をクラス内に閉じ込めることで、コードの見通しを良くしプログラムの全体像をシンプルに捉えることができます。

先の Person クラスでは、new したインスタンスのフィールドに直接値を代入・参照していました。

このクラスをカプセル化しましょう。

```
public class Person {
   private String name;
   private int age;

   // セッター
   public void setName(String name){
      this.name = name;
   }
   public void setAge(int age) {
      this.age = age;
   }
   // ゲッター
   public String getName() {
      return name;
   }
   public int getAge() {
      return age;
   }

   public void introduce(){
      if (name != null && age > 0) {
         System.out.println("My name is "+name+", My age is "+age+" years old.");
      }
   }
}
```

それぞれのフィールドに、private キーワードをつけてクラス内だけのアクセス可として、設定と取得のメソッドは public キーワードを使って公開メソッドにしました。

尚、メソッドの引数名とフィールド名が同じであるので、区別するためにインスタンスフィールド名の頭に this.XXXX と this キーワードを付けています。（this は自分自身のインスタンスを示す）

フィールドへの値の設定メソッドを setter（セッター）、値の取得メソッドのことを getter（ゲッター）といい、それぞれ通常は setXxxx(), getXxxx()という様に対応するフィールド名の頭文字を大文字にして前に set, get を付けた名前にします。

カプセル化したクラスを、外部から使った例を下記に示します。

```
class PersonTest {
   public static void main(String[] args) {
      Person abe = new Person(); // 阿部さんオブジェクトを作る
      abe.setName("Abe");// 阿部さんの名前を設定する
```

```
        abe.setAge(26);        // 阿部さんの年齢を設定する

        Person yamada = new Person(); // 山田さんオブジェクトを作る
        yamada.setName("Yamada"); // 山田さんの名前を設定する
        yamada.setAge(32);  // 山田さんの年齢を設定する

        System.out.println(abe.getName());
        System.out.println(abe.getAge());
        abe.introduce();
        System.out.println(yamada.getName());
        System.out.println(yamada.getAge());
        yamada.introduce();
    }
}
```

セッター／ゲッターのメリットとして引数のチェックなど機能追加があります。

例として、setAge(int age)の引数のチェックを追加してみましょう。

```
    public void setAge(int age) {
        if (age >= 0 && age <= 130) {
            this.age = age;
        }
    }
```

1.2.2. インナークラス（Inner Class）

あるクラス内だけで使うクラスを定義することが出来ます。このクラスを インナークラス（内部クラス: Inner Class）と呼びます。インナークラスを含む外側のクラスはアウタークラス(外部クラス: Outer Class)と呼びます。

クラス内部に定義するインナークラスには、protected、private、static を指定することが可能です。インナークラスは機能のカプセル化に貢献します。

```
class クラス名 {
    修飾子 class クラス名 {
        ...
    }
}
```

インナークラス／アウタークラスの例を下記に示します。

```
class Player {
    private String name;
    private int defensePower = 10; // プレイヤーの基本防御力

    public Player (String name) {
        this.name = name;
    }

    /**
     * インナークラス (Inner Class): SpecialArmor
```

```
     * インナークラスは、特定のクラス（この場合は Player）と密接に結びつき、
     * そのクラスのためだけに存在するヘルパーのような存在です。
     * * メリット：外部クラスの`private`メンバーに直接アクセスできます。
     */
    public class SpecialArmor {
        private String armorName = "聖なる鎧";
        private int armorBonus = 20;

        public void equip() {
            System.out.println(armorName + "を装備した！");
            // インナークラスから、外部クラス(Player)の private フィールド `defensePower` に直接アクセス！
            Player.this.defensePower += this.armorBonus;
        }

        public void displayPlayerStatus() {
            System.out.println(
                "[" + name + "] の現在の防御力は " + defensePower + " です。"
            );
        }
    }
}
```

1.3. コンストラクタとガベージコレクション

先ほどの例では、インスタンスを生成した後に内部でもつフィールドを初期化していました。もっとスマートに初期化する方法がコンストラクタと呼ばれる機能を用いることです。コンストラクタは、インスタンスの生成時に内部データの初期化を行うことで、外部から見ると new された後には準備が整ってそれを問題なく操作できるメリットがあります。

コンストラクタは、クラス名と同じ名前を持つ特別なメソッドです。インスタンスが生成された時に自動的に呼ばれ、インスタンスの初期化処理を行います。引数の無いコンストラクタはデフォルトコンストラクタといい、例えば new Person()としたときにインスタンスを生成するために呼ばれます。デフォルトコンストラクタはコンストラクタが一つも書かれていないクラスでも内部で存在します。
次の例では、Person クラスのインスタンス abe を生成する際に、コンストラクタの処理を利用して名前（name）の設定を行っています。引数付きのコンストラクタを記述した場合、デフォルトコンストラクタが必要であれば明示的に追加しなければいけません。引数なしでインスタンスを new しようとすると、デフォルトコンストラクタが無いためコンパイルエラーになります。

```
class Person {
   String name;
   Person(String name) {
      this.name = name;
   }
}
Person abe = new Person("abe"); // OK
Person yamada = new Person();          // NG
```

引数なしでインスタンスを生成したい場合、デフォルトコンストラクタも書くようにしましょう。

```
class Person {
   String name;

   Person() {};
   Person(String name) {
      this.name = name;
   }
   }
Person abe = new Person("abe");        // OK
Person yamada = new Person();          // OK
```

インスタンスが消滅する際に自動的に呼ばれる特別なメソッドをデストラクタと言い C++言語などではサポートされていますが、Java ではすべての破棄作業は自動的に行われるのでプログラムでのデストラクタは不要とされています。
しかしインスタンスが生成される**ヒープメモリ**上に多数の不要なインスタンスが残っているとメモリを圧迫するので、プログラム動作を監視・管理している JVM は自動的に適切なタイミングで解放します。JVM はヒープメモリが逼迫してくるとヒープの空き領域を確保するために、プログラム変数等で参照していない不要なインスタンスを見つけて解放処理を行います。この処理のことをガベージコレクション（ゴミ集め）といいます。ガベージコレクションは CPU などのコストの

高い処理で、プログラムの動作に悪影響を与えることもあります。従って、大量のインスタンスを作成するプログラムは注意が必要です。

ヒープメモリ(heap memory）：プログラム実行時に必要に応じてメモリを動的に確保・解放できる領域

クラス、インスタンス、フィールド、メソッドについて名称等を以下にまとめてみます。

用語	別名（意味）	具体例
クラス	インスタンスの雛形	Person
インスタンス	オブジェクト	abe, Yamada
コンストラクタ	インスタンスの初期処理	Person() Person(String name)
フィールド	属性 アトリビュート プロパティ	name, age
メソッド	関数 ファンクション	setName(), getName() setAge(), getAge()

2. 継承(extends)

2. 継承（extends）

2.1. スーパークラスとサブクラス

継承は、複数のオブジェクトが基本となる機能や性質を持つが、一部に機能の変更や追加が必要な場合、そのクラス群を作成したいときに役立ちます。
まずは共通する機能を持つベースとなるクラスを作り、それを親クラス（スーパークラスという）として、その機能を継承し、一部違う機能のクラスを子クラス（サブクラスという）として定義します。この継承のあるクラス同士は、「is a 関係」とも呼ばれます。つまり(子)は(親)の機能を持つことから“～は～の一種である”という関係が成り立つからです。
継承を使ったサブクラスの定義方法を下記に示します。

```
class サブクラス名 extends スーパークラス名 {
   …
}
```

下記の例では、前に作成した Person クラスを継承する Member クラスを定義しています。Member クラスは Person クラスを継承しているので、 name 、age などのフィールドや、 getName() 、setName() などのメソッドを引き継ぎ、加えて、myNumber や setNumber() などのフィールドやメソッドを備えています。

```
class Member extends Person {
   int number;
   public void setNumber(int number) {
      this.number = number;
   }
   public int getNumber() {
      return number;
   }
}
```

2.2. 自分自身（this）

クラスメソッドの中で this は自分自身を示す特別な変数名となります。下記の例では、this は、Person クラスのインスタンス自身を示します。

```
class Person {
   private String name;
   protected int age; // protected は自分またはサブクラスまでがアクセス可能

   Person(String name, int age) {
      this.name = name;
      this.age = age;
   }
   public void setName(String name) {
      this.name = name;
   }
   public String getName() {
      return name;
   }
}
```

2.3. スーパークラスの参照（super）

super は、クラスの継承における 親クラス（スーパークラス）を示します。java では、親クラスのコンストラクタは暗黙的には呼ばれないため、下記のように明示的に呼び出してやる必要があります。

```
class Member extends Person {
   private int number;
   void setNumber(int number) {
      this.number = number;
   }

   Member(String name, int age, int number) {
      super(name, age);    // 親クラスのコンストラクタを呼び出す
      this.setNumber(number);
   }
   public setAttributes(int name, String age, int number) {
      super.setName(name);          // 親のメソッド呼び出し
      super.age = age;              // 親のフィールドアクセス
      this.setNumber(number);
   }
}
```

2.3.1. 参照変数とキャスト

サブクラスのインスタンスを生成してその使用例を下記に示します。

```
public class App {
   public static void main(Strng[] args) {
      Member member1 = new Member("Jone", 22, 1);
      member1.getName();            // "Jone"
      member1.setAttributes("Jone", 25, 1);

      Person parson2 = new Member("Marry", 18, 2);
      parson2.getName();            // "Marry"
      parson2.setAttributes("Mearry", 25, 2); // ←コンパイルエラー
   }
}
```

Marry さん名前で Member クラスのインスタンスを生成して、そのスーパークラス Person クラス型の参照引数に入れることが出来ます。その参照変数から Person クラスにあるインスタンスメソッドだけ呼ぶ分は良いですが、Member クラスにだけあるインスタンスメソッドは呼び出すことは出来ません。

実態はサブクラスのインスタンスであっても参照変数の型がスーパークラスなので、スーパークラスの範囲の情報しかアクセスできていないからです。

一旦スーパークラス型の参照変数に入れたインスタンスをサブクラスに戻すには、次のように参照変数をキャストする必要があります。ただし、本来のインスタンスと無関係のクラスを指定してキャストを実行すると ClassCastException というエラーが発生するので注意が必要です。

```
キャスト文法：(キャストする型)参照変数
例：
・((Member)parson2).setAttributes(…..
```

```
・Member member2 = (Member)parson2;.
```

2.4. Object クラス

Object クラスとは全てのクラスの親クラス（ルートクラス）であり、Java の全てのクラス（配列を含む）が暗黙的に継承する基本クラスです。実は、継承の指定なく作成したクラスのスーパークラスも親は Object クラスで、new したインスタンスは全て Object 型の参照変数で受けらということです。

```
Object member1 = new Member(....);
Object person1 = new Person(....);
```

Object クラスの役割は、すべてのオブジェクトが持つべき基本的な機能を提供し、デフォルトコンストラクタや toString メソッド（文字列表現の取得）、equals メソッド（オブジェクトの同等性判定）、hashCode メソッド（それぞれのオブジェクトに固有のハッシュコードの取得）など、クラスとインスタンスの基本機能がこれにより実現されているのです。

2.5. クラスに属しているか調べる（instanceof）

instanceof 演算子は、オブジェクト（インスタンス）が、指定したクラスまたはその上位のクラスに属しているかどうかを調べます。

```
MyButton b = new MyButton();

if (b instanceof Button) {
   System.out.println("属してる");
} else {
   System.out.println("属していない");
}
```

３. ポリモーフィズム（多態性）

3. ポリモーフィズム（多態性）

ポリモフィズム（多態性）とは、異なるオブジェクトが共通のインターフェースやメソッド呼び出しに対して、それぞれ異なる振る舞いをする仕組みです。これにより、コードの再利用性や柔軟性が向上し、拡張性の高いプログラムを作成できます。ポリモフィズムは、主にメソッドのオーバーロードとオーバーライド、次節で述べられる、抽象クラスとインターフェースの活用によって実現されます。

3.1. オーバロード

Java では、同じ名前の変数を同一ブロック内で作れません。しかしメソッドに関しては、引数が違えば 1 つのクラスに同一名のメソッドを複数作ることが可能です。この機能をオーバーロードと呼びます。

オーバーロードの例を下記に示します。

```
public class Calculator{
   public static int add(int a, int b) {
      return a+b;
   }
   public static int add(int a, int b, int c) {
      return a+b+c;
   }
   public static double add(double a, double b) {
      return a+b;
   }

   public static void main(String[] args) {
      System.out.println(add(1, 2));
      System.out.println(add(1, 2, 3));
      System.out.println(add(3.2, 8.7));
   }
}
```

3.2. オーバライド

サブクラスをスーパークラス型の参照変数へ代入することが出来ます。

ただしこの場合、実体がサブクラスでも参照変数の型がスーパークラスになったのでサブクラスにだけあるフィールドやメソッドは使えません。

```
public class Car {
   …
}

public class ElectricCar extends Car {
   boolean isERS;// Energy-Recovery System (ERS) - 「回生充電システム」
   void setERS(Boolean isERS) { … }
}

public class App {
      public static void main(String[] args) {
         Car car = new Car();
         Car car2 = new ElectricCar();
         // System.out.println(car2.isERS);      ←使用できない！
```

```
        // car2.setERS(true);                    ←使用できない！
```

サブクラスに、スーパークラスにないフィールドやメソッドを定義することにより新たな機能を追加できます。また、スーパークラスの機能に存在するメソッドと同名メソッドを定義して処理上書き定義することが出来ます。

同じというのはメソッド名、引数、戻り値の型が全て同じということです。これをオーバーライドと言います。

```
public class Car {
   void fullCharge() {
      System.out.println("ガソリンを入れる");
   }
}

public class ElectricCar extends Car {
   void fullCharge() {
      System.out.println("充電する");
   }
}


public class App {
      public static void main(String[] args) {
         Car car = new Car();
         car.fullCharge();              // "ガソリンを入れる"
         Car car2 = new ElectricCar();
         car2.fullCharge();             // "充電する"
```

3.3. 抽象化

抽象化とは、複雑な内部の仕組みを隠し、利用者に必要最低限の機能（使い方）だけを見せるという設計思想です。利用者は「何ができるか」だけを意識すればよく、「どうやってそれを実現しているか」を知る必要はありません。Java における抽象化では、大きく抽象クラスとインターフェースという二つの手法が用意されています。
どちらも外部から呼ぶことのできるメソッド名や引数、戻りの型を定義して、処理の中身は実装クラスに委ねられます。
抽象クラスは、共通の機能や性質を持つが一部の具体的な動作が異なるクラス群の「ひな形」としてのスーパークラスに使われます。abstract キーワードで抽象メソッドを定義して、実装は抽象クラスを継承したサブクラスに委ねられます。
インターフェースは、クラスが実装すべき「機能の契約（ルール）」だけを定義するものです。それぞれのインターフェースにはその役割に応じて、まとまった複数の抽象メソッドを定義できます。実装するクラスは、implement interface 名 1, interface 名 2, ….というように複数のインターフェースを実装できます。
従って、抽象化を考える際には「～は～の一種である（is-a）」関係なら抽象クラス、「～の機能を持つ（has-a）」関係ならインタフェースといった使い分けができます。

3.4. 抽象クラス

クラスに、処理の実体のない抽象メソッドと呼ばれる定義だけのメソッドを書くことが出来ます。
その様なクラスは抽象クラスと呼ばれ抽象メソッドが少なくとも 1 つあります。

```
abstract class クラス名 {
   abstract 型 メソッド名();
}
```

例えば 抽象クラスとして Fruit クラスを作成し共通の果物の色や価格を返すメソッドを定義します。Fruit クラスは特有の色や価格は未定なので特定できないため抽象メソッドとして定義しておきます。具体的な果物は、Fruit クラスをスーパークラスとしてそれを継承したサブクラス作り抽象メソッドをオーバーライドします。以下にこの具体例を示します。

```
public abstract class Fruit {
   abstract String getColor();
   abstract int getPrice();
}

public class Banana extends Fruit {
   String getColor() {
      return “yellow”;
   }
   int getPrice() {
      return100;
   }
}
public class Strawberry extends Fruit {
   String getColor() {
      return “red”;
   }
   int getPrice() {
      return200;
   }
```

```
}

public void App {
   public static void main(string[] args) {
      Banana banana = new Banana();
      System.out.println("Banana:"+banana.getColor()+","+banana.getPrice()+"円");
      Strawberry strawberry = new Strawberry();
      System.out.println("Strawberry:"+ strawberry.getColor()+","+ strawberry.getPrice()+"円");
   }
}
```

抽象クラスを継承して定義されたクラスを具象クラスと言います。具象クラスは抽象クラスの抽象メソッドを必ずオーバーライドしなければなりません。

3.5. インタフェース（interface）

インタフェースは、ある機能を実現するクラスが必ず実装するべきメソッドの名前や型といった作法のみを定義するものです。 interface を用いて定義します。インタフェースで定義されるメソッド（抽象メソッド）の実体は implements を用いた側のクラスで実装しなくてはなりません。

```
interface インタフェース名 {
   ...
}
```

下記は ActionListener インタフェースの実際の定義例です。 EventListener インタフェースを継承しています。インタフェース内で定義されるメソッドには、暗黙的に abstract 宣言と public 宣言がつけられます。

```
public interface ActionListener extends EventListener {
   public void actionPerformed(ActionEvent e);
}
```

3.5.1. インタフェースの実装（implements）

インタフェースを 実装するには、implements を用いて下記のように定義します。インタフェース名は、カンマ（,）で区切って複数指定することができます。implements を用いた場合、インタフェース名で指定された抽象メソッドを実装することが義務付けられます。

```
class クラス名 implements インタフェース名 {
   ...
}
```

インタフェースの例

インタフェースの具体例を見てみましょう。InterfaceA はテスト用のインタフェースです。メソッド methodA()と methodB()の名前や型のみを定義しています。クラス ClassA は InterfaceA を実装すると宣言しているため、methodA()や methodB()を正確に実装しなくてはなりません。この実装を行う際のプログラミングミスをチェックするのが、インタフェースの役目です。

```
interface InterfaceA {
   public abstract void methodA(String msg);
   public abstract void methodB(int x);
}

class InterfaceTest implements InterfaceA {
   public static void main(String[] args) {
      InterfaceTest o = new InterfaceTest();
      o.methodA("ABC");
      o.methodB(123);
   }
   public void methodA(String msg) {
      System.out.println(msg);
   }
   public void methodB(int x) {
      System.out.println(x);
   }
}
```

3.5.2. インタフェース修飾子（public, ...）

インタフェースの修飾子には、public を指定することができます。

```
修飾子 interface インタフェース名 {
   ...
}
```

インタフェースをクラス内に定義する場合には加えて、protected、 private、static を指定することが可能です。

```
class クラス名 {
   修飾子 interface インタフェース名 {
      ...
   }
}
```

3.5.3. インタフェースの継承（extends）

親インタフェースの定義を継承するには extends を用います。

```
interface インタフェース名 extends 親インタフェース名 {
   ...
}
```

3.5.4. default と static メソッド [Java8～]、private メソッド[Java9～]

インターフェースはクラスへ実装することを前提に、中身の書かれていない抽象メソッドを用意していましたが、Java8 からインターフェースに新機能として default メソッドと static メソッドが実装されたことにより、インターフェース自体が中身のある具象メソッドを持てるようになりました。尚、default メソッドのアクセス修飾子は暗黙的に public です。

```
public interface SampleInterface {
   default String get() {
      return "default method call";
   }
```

```
}
public class Sample implements SampleInterface {
}
public class App{
   public static void main(String[] args) {
      DefaultTestSample sample = new Sample(); // インスタンス化
      System.out.println(sample.get()); // default method call
   }
}
```

２つ以上のインターフェースに同名の default メソッドがあった場合、メソッド名だけではどちらが呼ぶべきか JVM は分からないのでエラーになります。その様な場合の“インターフェース名.super.メソッド名“でコールするメソッドを指定します。下記に例を示します。

```
public class App{
   public interface SampleInterface {
      default String get() {
         return "Base method call";
      }
   }

   public interface SampleInterfaceA extends SampleInterface{
      default String get() {
         return "A method call";
      }
      default String getBase() {
         return SampleInterface.super.get();
      }
   }

   public interface SampleInterfaceB extends SampleInterface{
      default String get() {
         return "B method call";
      }
   }

   public static class Sample implements SampleInterfaceA, SampleInterfaceB {
      public String get() {
         return SampleInterfaceA.super.get();
      }
      public String getB() {
         return SampleInterfaceB.super.get();
      }
   }

   public static void main(String[] args) {
      Sample sample = new Sample(); // インスタンス化
      System.out.println(sample.get()); // A method call
      System.out.println(sample.getB()); // B method call
      System.out.println(sample.getBase()); // Base method call
   }
```

```
}
```

インターフェースに static キーワードを付けて処理内容を書くことができます。

また、実装クラスからは呼べない private キーワードを付けて処理内容を書けるようになりました。

static メソッドは、インスタンス生成をしなくても、クラスから直接呼び出すことが可能となります。

```
public class App{
  public interface StaticMethodInterface {
    // static メソッド
    static String get() {
      return "My name is " + getMyName();
    }
    // private 付き static メソッド
    private static String getMyName() {
      return "StaticMethodInterface";
    }
  }

  public static void main(String[] args) {
    System.out.println(StaticMethodInterface.get()); // My name is StaticMethodInterface
  }
}
```


```
public class App{
  public interface StaticMethodInterface {
    // static メソッド
    static String get() {
      return "My name is " + getMyName();
    }
    // private 付き static メソッド
    private static String getMyName() {
      return "StaticMethodInterface";
    }
  }

  public static void main(String[] args) {
    System.out.println(StaticMethodInterface.get()); // My name is StaticMethodInterface
  }
}
```

４. パッケージとアクセス修飾子

4. パッケージとアクセス修飾子

パッケージはクラスのまとまり、アクセス修飾子はクラスやメンバーの公開範囲を決定するキーワードです。
どちらも自分が作ったクラス等を、どこまで他に公開するか？または他者が作ったクラスを使用するために取り込むかを決定するものです。

4.1. インポート（import）

Java において、例えば JDK にある日付クラスライブラリは LocalDateTime ですが、そのパッケージは、java パッケージの下に、time パッケージがありその中にあります。
従って正式なパッケージ付きのクラス名は住所を示すように、“.”をつなげて java.time.LocalDateTime となります。
しかし、毎回 java.time.LocalDateTime と記述するのは面倒なため、プログラムの冒頭で java.time.LocalDateTime を import しておくことにより、java. time.LocalDateTime を単に LocalDateTime と記述できるようになります。
import java.time.LocalDateTime;と import java.time.format.DateTimeFormatter;を、まとめて import java.time.*;（”*”：アスタリスクは全ての意味）とも記述できます。

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
public class PakageA {
   public static void main(String[] args) {
      DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm:ss");
      java.time.LocalDateTime time1 = LocalDateTime.now(); // java.time.LocalDateTime フルで記述
      System.out.println(time.format(formatter));
      LocalDateTime time2 = LocalDateTime.now(); //LocalDateTime だけで記述
      System.out.println(time.format(formatter));
   }
}
```

```
import java.time.*;
public class PakageB {
   public static void main(String[] args) {
      DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm:ss");
      LocalDateTime time1 = LocalDateTime.now();
      System.out.println(time.format(formatter));
      LocalDateTime time2 = LocalDateTime.now();
      System.out.println(time2.format(formatter));
   }
}
```

4.2. パッケージ文（package）

パッケージ は、いくつかのクラスやインタフェースをひとまとまりにし名前を付けたものです。ファイルの先頭で package 文を次のように宣言することで、そのファイルに含まれるクラスなどは、pkgA.pkgB パッケージの一部として定義されるようになります。

```
package pkgA.pkgB;
```

パッケージのサンプル

下記のようなフォルダ構成でファイルを作成してみます。□はドライブやフォルダ、○はファイルを意味しています。

□ C:/
└□ java
├○ App.java
└□ pkgA
├○ PkgTest2.java
└□ pkgB
└○ PkgTest3.java

それぞれのファイルの中身を次のように記述してください。

```
class App {
  public static void main(String[] args) {
    pkgA.PkgTest2 o2 = new pkgA.PkgTest2();
    pkgA.pkgB.PkgTest3 o3 = new pkgA.pkgB.PkgTest3();
    o2.test(); // this is pkgA.PkgTest2
    o3.test(); // this is pkgA.pkgB.PkgTest3
  }
}
```

```
package pkgA;
  public class PkgTest2 {
    public void test() {
      System.out.println("this is pkgA.PkgTest2");
      }
  }
}
```

```
package pkgA.pkgB;
  public class PkgTest3 {
    public void test() {
      System.out.println("this is pkgA.pkgB.PkgTest3");
    }
  }
}
```

4.3. アクセス修飾子

クラス、フィールド、メソッド、コンストラクタには、アクセス修飾子を指定することが出来ます。これにより自分のクラス以外に対象を公開するか・継承を許すか・異なるパッケージに対しても公開するかなど制約を定義できます。

4.3.1. クラスの修飾子

修飾子	説明
public	このクラスはどこからでも参照可能
abstract	このクラスは抽象クラスです
final	このクラスから継承することができない
ない	このクラスは同じパッケージからしか参照できない（パッケージプライベート）

4.3.2. フィールドの修飾子

修飾子	説明
public	このフィールドはどこからでも参照可能
protected	このフィールドは同じパッケージか、このクラスを継承したクラスからしか参照できない
ない	このフィールドは同じパッケージからしか参照できない（パッケージプライベート）
private	このフィールドは自分のクラス内からしか参照できない
final	このフィールドは初期値から変更できない
static	このフィールドは静的フィールドです

4.3.3. メソッドの修飾子

修飾子	説明
public	このメソッドはどこからでも参照可能
protected	このメソッドは同じパッケージか、このクラスを継承したクラスからしか参照できない
ない	このメソッドは同じパッケージからしか参照できない（パッケージプライベート）
private	このメソッドは自分のクラス内からしか参照できない
final	このメソッドは（サブクラスで）オーバーライドできない
abstract	このメソッドは抽象メソッドです
static	このメソッドは静的メソッドです（クラスメソッドで、インスタンス不要）
synchronized	このメソッドは**スレッドセーフ**（排他制御）であることを示す

スレッドセーフ：複数の処理単位であるスレッドが同時にデータを共有・利用しても、データの整合性が保たれるような設計・実装であること。マルチスレッド環境において、共有データへのアクセスが競合して予期せぬ不具合（競合状態やデータ不整合）を引き起こすのを防ぎ、プログラムが常に正しい結果を返すようにする。

4.3.4. コンストラクタの修飾子

修飾子	説明
public	このコンストラクタはどこからでも参照可能
protected	このコンストラクタは同じパッケージか、このクラスを継承したクラスからしか参照できない
ない	このコンストラクタは同じパッケージからしか参照できない
private	このコンストラクタは自分のクラス内からしか参照できない

5. 変数とその扱い

5. 変数とその扱い

5.1. 基本型とラッパークラス

変数の基本型に対して、Java ではラッパークラスと呼ばれる対応するクラスがあります。

なぜその様なものがあるかというと、基本型にはない機能を有するためにクラスで値を保持することにより便利なメソッド機能が使え、他の一般的なクラスのインスタンスと統一的な処理ができるなどの利点があります。

各基本型に対応するラッパークラスを下記に示します。

基本型	ラッパークラス
boolean	Boolean
byte	Byte
char	Character
double	Double
float	Float
int	Integer
long	Long
short	Short

以前は基本型とラッパーオブジェクトの変換には特別なメソッドが必要でしたが、現在は下記のように単純に"="で代入することで相互変換可能です。

```
int a = 10;
Integer b = a;
int c = b;
a = b + c;

//以前の変換方法
int a = 10;
Integer b = Integer.valueOf(a);
int c = b.intValue();
a = b.intValue() + c;
```

文字列からラッパーオブジェクトを作成するには、下記の valueOf メソッドが使えます。

```
Integer i = Integer.valueOf("123");
Long l = Long.valueOf("123");
Double d = Double.valueOf("3.14");
Integer hex = Integer.valueOf("FF", 16); // 16 進数の文字列表現"FF"で Integer 型に変換する
```

5.2. 数値演算（Math クラス、Random クラス）

java.lang.Math クラスには、クラスに有用な数値演算メソッドがあります。主なメソッドを下記に示します。

```
import java.lang.Math;

//絶対値
```

```
// public static T abs(T a)
// T：数値型（int、long、double、float）、a：任意の数値
System.out.println(Math.abs(-20.3));    // 20.3
System.out.println(Math.abs(-20.3));    // 20.3

//数の切り上げ、切り捨て、四捨五入
// public static double ceil(double a) - 切り上げ
// public static double floor(double a) -  切り捨て
// public static int round(float a) - 四捨五入
// public static long round(double a) - 四捨五入
// a：任意の値
System.out.println(Math.ceil(3.5));   // 4.0
System.out.println(Math.floor(5.5)); // 5.0
System.out.println(Math.round(3.5));      // 4
System.out.println(Math.round(3.4));      // 3

// 2つの値のうち大きい方を返す
// public static T max(T a, T b)
// T：数値型（int、long、double、float）、a, b：任意の数値
System.out.println(Math.max(1, 7)); // 7


// 2つの値のうち小さい方を返す
// public static T min(T a, T b)
// T：数値型（int、long、double、float）、a, b：任意の数値
System.out.println(Math.min(1, 7)); // 1

// double値の乱数（0.0以上で1.0より小さいdouble値）を返します。
// public static double random()
System.out.println(Math.floor(Math.random()*10); // 0から9までの整数値を返します
```

他に正の整数で指定範囲の乱数を生成する場合は、java.util.Random クラスの nextInt() を使います。

```
import java.util.Random;

Random rand = new Random();
int num = rand.nextInt(10); // nextInt() は 0 から引数に指定した値未満の整数を返します
System.out.println(num);
```

5.3. 日付／時刻（Date-Time API）[Java8～]

日付／時刻を扱う新しい API として、java.time パッケージの Date and Time API が導入されました。

古い日付クラスとして、Date クラスがありますが、Date クラスはミュータブル（可変）でスレッドセーフではありません。また、ミリ秒単位の精度しか提供しておらず時区情報もありません。従ってこれらの欠点から、新しい日時 API を使用することが推奨されています。

java.time パッケージで提供されている主なクラスは以下です。

・LocalDateTime、LocalDate、LocalTime：時差情報を持たないローカル日時

・OffsetDateTime、OffsetTime：時差情報を持つ日時

・ZonedDateTime：タイムゾーンと地域に基づいた情報（サマータイム）を持つ日時

現在時刻は、java.time.LocalDateTime の now メソッドを用いて取得します。

java.time.LocalDateTime オブジェクトの値は、1970 年 1 月 1 日 0 時からの経過時間をナノ秒で表したものです。

```
import java.time.LocalDateTime;

LocalDateTime local = LocalDateTime.now(); //  現在時刻の取得
int year = local.getYear();                // 年を取得
int month = local.getMonthValue();         // 月を取得
int day = local.getDayOfMonth();           // 日を取得
int hour  = local.getHour();               // 時を取得
int minute = local.getMinute();            // 分を取得
int second= local.getSecond();             // 秒を取得

System.out.println(local); // 2025-09-25T18:30:44.577135900
OffsetDateTime offset = OffsetDateTime.of(2025, 9, 25, 18, 15, 30, 0, ZoneOffset.of("+09:00"));
System.out.println(offset);// 2025-09-25T18:15:30+09:00
ZonedDateTime zoned = ZonedDateTime.parse("2025-09-25T18:15:30+09:00[Asia/Tokyo]");
System.out.println(zoned); // 2025-09-25T18:15:30+09:00[Asia/Tokyo]

// フォーマッターの作成と活用
// 例: yyyy/MM/dd
DateTimeFormatter formatter1 = DateTimeFormatter.ofPattern("yyyy/MM/dd");
String formattedDate1 = local.format(formatter1);
System.out.println("yyyy/MM/dd: " + formattedDate1); // 2025/09/20

//例: yyyy 年 MM 月 dd 日 HH 時 mm 分
DateTimeFormatter formatter2 = DateTimeFormatter.ofPattern("yyyy 年 MM 月 dd 日 HH 時 mm 分");
String formattedDate2 = local.format(formatter2);
System.out.println("yyyy 年 MM 月 dd 日 HH 時 mm 分: " + formattedDate2); // 2025 年 09 月 20 日 10 時 30 分

// ISO 形式のフォーマットパターン (例: yyyy-MM-ddTHH:mm:ss)
DateTimeFormatter formatter3 = DateTimeFormatter.ISO_LOCAL_DATE_TIME;
String formattedDate3 = local.format(formatter3);
System.out.println("ISO_LOCAL_DATE_TIME: " + formattedDate3); // 2025-09-20T10:30:00
```

5.4. 文字列

5.4.1. 文字列（String）

文字列 を扱うには、String を用います。

```
String str = "abc";
```

5.4.2. 文字列長を得る（length, isEmpty）

length() は、文字列の長さを返します。Java では Unicode を基本とするため、日本語のような全角文字も 1 文字として数えます。 isEmpty は、文字列の長さが 0 つまり空であるかどうかを調べます。

```
String str = "ABCDEFG";
System.out.println(str.length());
System.out.println(str.isEmpty());
str = “”;
System.out.println(str.isEmpty());
```

5.4.3. 等しいかどうか調べる（equals, equalsIgnoreCase）

equals(str) は、文字列が str と等しいかどうかを調べます。

```
String str = "ABC";
if (str.equals("ABC")) {
  ...
}
// 大文字・小文字を無視して比較する場合は equalsIgnoreCase(str) を用います。
String str = "ABC";
if (str. equalsIgnoreCase("abc")) {
    ...
}
```

文字列を比較する際に、== 演算子を用いてはなりません。strA== strB は、strA と strB が同じオブジェクトかどうかを調べたりする際に用いる演算子です。例えば、下記の例では、strA も strB も文字列の値は "ABCDEF" のはずですが、演算結果は false になります。

```
String strA = "ABC";
String strB = "ABC";
strA = strA + "DEF";
strB = strB + "DEF";
if (strA == strB) {
    System.out.println("true");
} else {
    System.out.println("false");
}
```

5.4.4. 文字列を整形する（format）

文字列を指定された書式文字列で整形します

public static String format([Locale locale,] String format, Object… args)

locale：整形に利用するロケール（省略時はシステムデフォルト）、format：書式文字列、args：書式に割り当てる値

書式文字列は、次のような構成要素を順番に指定します。

%[インデックス$][フラグ][幅][.精度]型

<table>
<tr><th>指定子</th><th>概要</th></tr>
<tr><td>インデックス</td><td>引数リスト内での引数の位置</td></tr>
<tr><td>フラグ</td><td>出力形式を指定</td></tr>
<tr><td>幅</td><td>出力する最小の文字数</td></tr>
<tr><td>精度</td><td>全体桁数（整数）、または小数点以下の桁数（浮動小数点数）</td></tr>
<tr><td>型</td><td>値の型
<table>
<tr><th>型</th><th>概要</th></tr>
<tr><td>b、B</td><td>ブール値</td></tr>
<tr><td>h、H</td><td>ハッシュコードを 16 進数に変換</td></tr>
<tr><td>s、S</td><td>文字列</td></tr>
<tr><td>c、C</td><td>Unicode 文字</td></tr>
<tr><td>d</td><td>10 進整数</td></tr>
<tr><td>o</td><td>8 進整数</td></tr>
<tr><td>x、X</td><td>16 進整数</td></tr>
<tr><td>e、E</td><td>浮動小数点数形式の 10 進数</td></tr>
<tr><td>f</td><td>10 進小数点数</td></tr>
<tr><td>t、T</td><td>
<table>
<tr><th>書式</th><th>概要</th></tr>
<tr><td>Y</td><td>年（4 桁）</td></tr>
<tr><td>m</td><td>月（01～12）</td></tr>
<tr><td>d</td><td>日（01～31）</td></tr>
<tr><td>H</td><td>時（00～23）</td></tr>
<tr><td>I</td><td>時（01～12）</td></tr>
<tr><td>M</td><td>分（00～59）</td></tr>
<tr><td>S</td><td>秒（00～59）</td></tr>
<tr><td>L</td><td>ミリ秒（000～999）</td></tr>
<tr><td>D</td><td>日付（%tm/%td/%ty と同じ）</td></tr>
</table></td></tr>
</table></td></tr>
</table>

		T	時刻（%tH:%tM:%tS と同じ）
		日付／時刻変換文字の接頭辞	
	%	リテラル”%”（u0025）	
	n	プラットフォーム固有の行区切り文字	

下記に例を示します。

```
String str = "good morning";
System.out.println(String.format("-- %4s --」", str)); // -- good --
double num = 3.141592;
System.out.println(String.format("値：%f です。", num));    // 値：3.141592」です。
System.out.println(String.format("値：%.2f です。", num));      // 値：3.14 です。
System.out.println(String.format("値：%e です。", num));     // 値：「3.141592e+00」です。
System.out.println(String.format("値： %E です。", num));     // 値：「3.141592e +00」です。

Date c = new Date();
System.out.println(String.format("%tY 年%tm 月%td 日 %tH 時%tM 分%tS 秒", c, c, c, c, c, c)); // 2025 年 10 月 22 日 14 時 23 分 36 秒
```

5.4.5. その他メソッド

・文字列を大小比較する（compareTo, compareToIgnoreCase）

compareTo(str) は、文字列が、str と比べて辞書的にどちらが大きいか調べます。str のほうが大きい時は負の値を、等しい時は 0 を、小さい時は正の値を返します。

```
String str1 = "ABC";
String str2 = "DEF";
if (str1.compareTo(str2) == 0) { ... }
// 大文字・小文字を無視して比較する場合は compareToIgnoreCase(str) を用います。
String str1 = "ABC";
String str2 = "abc";
if (str1.compareToIgnoreCase(str2) == 0) { ... }
```

・n 番目の文字を得る（charAt）

charAt(n) は、0 を基点として n 番目の文字を返します。

```
String str = "ABCDEFG";
System.out.println(str.charAt(2));
```

・指定文字が出現する場所を得る（indexOf, lastIndexOf）

str.indexOf(ch) は、文字列の中で文字 ch が最初に出現する位置を、0 を基点とする文字数で返します。見つからない時は -1 を返します。

```
String str = "ABCDEFG";
```

```
System.out.println(str.indexOf('C'));
// 文字の代わりに文字列を指定することもできます。
String str = "ABCDEFG";
System.out.println(str.indexOf("DEF"));
// 第 2 引数 n を指定すると、n 番目以降の場所から探します。
String str = "ABCABCABC";
System.out.println(str.indexOf('A', 2));
// lastIndexOf(ch) は、文字列の後ろから探します。
String str = "ABCABCABC";
System.out.println(str.lastIndexOf('A'));
```

その他にも有用なメソッドがあります。

```
// 文字列の前後の空白を除去する
// void trim()
String str = "  good morning ! ";
System.out.println(str.trim()); // good morning !


String url = "http://www.aaa.com/index.html";
// 文字列が指定された部分文字列で始まるか？
// boolean startsWith(String prefix [,int offset])
// prefix：接頭辞、offset：比較を開始する文字位置
System.out.println(url.startsWith("http://")); // true

// 文字列が指定された部分文字列で終わるか？
// boolean endsWith(String suffix)
// suffix：接尾辞
System.out.println(url.endsWith(".net")); // false

 String str = "Hello";
// 文字列(半角英字)を小文字に変換する
// public String toLowerCase([Locale locale]) -
// locale：整形に利用するロケール（省略時：システムデフォルト）
System.out.println(str.toUpperCase()); // HELLO

// 文字列(半角英字)を大文字に変換する
// public String toUpperCase([Locale locale])
// locale：整形に利用するロケール（省略時：システムデフォルト）
System.out.println(str.toLowerCase()); // hello

// 部分文字列を取得する
// public String substring(int begin [,int end])
// begin：開始位置、end：終了位置
String str = "Good Morning Mr.Mark.";
System.out.println(str.substring(5));  // Morning Mr.Mark..
System.out.println(str.substring(5, 12));  // Morning


// 文字列を置き換える
// public String replace(char old, char new)
```

```
// public String replace(CharSequence old, CharSequence new)
// old：置換対象の文字（列）、new：置換後の文字（列）
String str = "Good Morning Everyone.";
System.out.println(str.replace("Morning", " afternoon")); // Good afternoon Everyone.

// 文字列を分割する
// public String[] split(String regex [,int limit])
// regex：区切り文字（正規表現）、limit：最大の分割数
String str = "2025/06/21";
String[] array = str.split("/");
for (String s : array) {
   System.out.println(s);
}

// 文字列を連結する
// public static String join(CharSequence deli, CharSequence… elems)
// public static String join(CharSequence deli,Iterable elems)
// deli：区切り文字、elems：連結する文字列
String str = String.join("/", "2025", "06", "21");
System.out.println(str); // "2025/06/21"
```

5.4.6. StringBuffer と StringBuilder

Java には String クラスの他に文字列を扱う StringBuffer クラスと StringBuilder クラスがあります。

String クラスと同様なメソッドを使えますが、文字列処理の状況に合わせてクラスを選択する必要があります。

String クラスでは、そのオブジェクトに何らかの加工をした場合、全く新しいオブジェクトが new される所謂、不変(**イミュータブル**)なオブジェクトです。

イミュータブル（immutable）：「不変」「変更不能」を意味し、一度作成されたデータやオブジェクト、システムの状態が後から変更できないことを指します。

```
String str1 = “hello”;
String str2 = str1;
String str3 = “ world”;
str1 = str1 + str3;
System.out.println(‘str1:” + str1);  // hello world
System.out.println(‘str2:” + str2);  // hello
System.out.println(‘str3:” + str3);  // world
System.out.println(‘str1 == str2 ?” + (str1 == str2)); // false
```

String オブジェクトでは、文字列演算のたびに新しいオブジェクトを作るため、メモリの確保と過去に使ったオブジェクトメモリの開放を行うことから、頻繁に演算を行う場合アプリケーションのパフォーマンス悪くなります。

それに対して、StringBuffer や StringBuilder クラスのオブジェクトでは、文字列演算をしても最初に作成されたオブジェクトはそのままで文字列の長さに従って内部に持っているバッファサイズが更新されるだけです。

```
StringBuffer sb1 = new StringBuffer("hello");
StringBuffer sb2 = sb1;
sb1.append(" world");
System.out.println(‘sb1:” + sb1);  // hello world
System.out.println(‘sb2:” + sb2);  // hello world
```

```
System.out.println('sb1 == sb2 ?" + (sb1 == sb2)); // true
```

従って、頻繁に演算を行う文字列は String ではなく StringBuffer または StringBuilder を使うと良いでしょう。
あとは、StringBuffer または StringBuilder どちらを選ぶかですが、StringBuffer または StringBuilder はどちらも可変サイズのバッファを持っている特徴があり、提供しているメソッドも同じです。
ただし、StringBuffer はスレッドセーフ（マルチスレッドでも安全）で、StringBuilder はスレッドセーフではなくシングルスレッド環境では若干速くなります。従って、マルチスレッド環境が多い今日では処理速度を厳しく求められる場合でない限り、安全のため StringBuffer を選択すると良いでしょう。

5.5. 配列（array）

5.5.1. 配列の作成

次のようにして、3 個の要素からなる配列を生成することができます。

```
class Test {
  public static void main(String[] args) {
    int[] xx = new int[3];
    xx[0] = 100;
    xx[1] = 200;
    xx[2] = 300;
  }
}
```

次のようにして、生成と初期化を一度に行うこともできます。

```
int[] xx = { 100, 200, 300 };
```

配列の個数は length で参照できます。

```
int[] xx = new int[3];
int n = xx.length;
```

二次元等多次元の配列を生成することも可能です。

```
int[][] xx = new int[3][5];
String[][] ss = {
  { "Sun", "Sunday" },
  { "Mon", "Monday" }
}
```

オブジェクトの配列を生成するには、配列生成後、それぞれの要素も個別に生成する必要があります。

```
ClassA[] xx = new ClassA[3];
xx[0] = new ClassA();
xx[1] = new ClassA();
xx[2] = new ClassA();
```

5.5.2. java.util.Arrays クラス

java.util パッケージにある Arrays クラスには、配列を操作する様々な便利な機能があります。

機能	説明
初期化	fill()
	int[] arr = new int[5]; Arrays.fill(arr, 1); System.out.println(Arrays.toString(arr));　// → [1, 1, 1, 1, 1]
リスト変換	asList()
	String[] fruits = {"Apple", "Banana", "Orange"}; List<String> fruitList = Arrays.asList(fruits); System.out.println(fruitList);　// [Apple, Banana, Orange]
コピー	copyOf(), copyOfRange()

	int[] original = { 10, 20, 30, 40, 50 };
	int[] copy = Arrays.copyOf(original, original.length);
	System.out.println(Arrays.toString(copy)); // [10, 20, 30, 40, 50]
	int[] copy2 = Arrays.copyOf(original, 3);
	System.out.println(Arrays.toString(copy2)); // [10, 20, 30]
	int[] copy3 = Arrays.copyOf(original, 7); //元の配列では有効でないインデックスがある場合、コピーには(byte)0 が含まれます。
	System.out.println(Arrays.toString(copy3)); // [10, 20, 30, 40, 50, 0, 0]
	int[] partCopy = Arrays.copyOfRange(original, 1, 4); //終端インデックスはコピーに含まない
	System.out.println(Arrays.toString(partCopy)); // [20, 30, 40]

比較	equals()
	int[] arr1 = { 1, 2, 3 }; int[] arr2 = { 1, 2, 5 }; System.out.println(Arrays.equals(arr1, arr2)); // false
表示	toString()
	int[] numbers = { 10, 20, 30, 40 }; System.out.println(Arrays.toString(numbers)); // [10, 20, 30, 40]
検索	binarySearch()

	int[] numbers = { 10, 20, 30, 40, 50 };
	int index = Arrays.binarySearch(numbers, 30);
	System.out.println("30 の位置：" + index); //　30 の位置：2

ソート	sort()
	int[] numbers = { 10, 20, 30, 40 }; System.out.println(Arrays.toString(numbers)); // [10, 20, 30, 40]

5.6. 列挙型（Enum 型）

enum キーワードを使用し、enum ブロックに定数名をカンマ区切りで列記します。列挙型は定数と同じ扱いなので、名前はアンダースコア形式（すべて大文字、単語の区切りはアンダースコア）で表すのが一般的です。

列挙型は定数なので、new などせずに WeekDay.SUNDAY の様に enum 型名.定数でアクセス可能です。

```
public enum WeekDay {
  SUNDAY, MONDAY, TUESDAY,
      WEDNESDAY, THURSDAY, FRIDAY, SATURDAY; }

System.out.println(WeekDay.SUNDAY.ordinal());

for (WeekDay day : WeekDay.values()) {
  System.out.println(day.name());
}
```

Enum 型には、次のメソッドが使えます。

```
String name()：列挙定数の名前を取得
int ordinal()：列挙定数の順序を取得（0 Origin）
String toString()：列挙定数の名前を取得（Override 可能メソッド）
E[] values()：すべての列挙定数を取得
```

Enum に短縮名称を付けて名前を取得したい場合など、次の様にフィールドを追加する事もできます。（名前でなく数値にしたい場合も同様）

```
public enum WeekdayEnum {
  SUNDAY("sun"),
  MONDAY("mon"),
  TUESDAY("tue"),
  Wednesday("wed"),
  WEDNESDAY("thu"),
  FRIDAY("fri"),
  SATURDAY("sat");

  private String nameShort;

  // コンストラクター（public でなく private であることに注意！）
  private WeekdayEnum(String name) {
    this.name = nameShort;
  }

  // メソッド
  @Override
  public String toString() {
    return this. nameShort;
  }

System.out.println(WeekdayEnum.WEDNESDAY.toString());
```

6. コレクション（集合）

6. コレクション（集合）

6.1. コレクションフレームワーク

コレクション（コレクションフレームワーク）とは、要素オブジェクトの集合を扱うための仕組みです。要素の集合の持ち方に様々なタイプがあります。

コレクションを使用する際は、あとで示すジェネリクスを積極的に使用して下さい。

例：ArrayList<String>の様に、<…>の形式で要素の型を明示します。

カテゴリ	インタフェース/クラス	説明
Collection	java.util.Collection	下記のコレクションの基底インタフェースで、下記の様な抽象メソッド定義がなされています。 add(Object o), remove(Object o), contains(Object o), size(), isEmpty()
List 系	java.util.List	Collection を親にもつ順序付きインタフェース List 特有の抽象メソッドがいくつか追加されています。 add(int index, Object o), get(int index), set(int index, Object element) , indexOf(Object o), lastIndexOf(Object o), subList(int fromIndex, int toIndex)
	ArrayList	要素を順番に並べた配列クラス 要素は、順位を示す index を示して取得・挿入・削除します。 配列から List に変換する場合は、java.util.List の toArray(Object[])メソッドを使用 List<String> list = new ArrayList<String>(Arrays.asList("Apple", "Orange", "Melon")); String[] array = (String[]) list.toArray(new String[list.size()]);
	LinkedList	要素の前・後ろを示すリンク付き配列クラス その為、挿入・削除が高速です。 要素は、順位を示す index を示して取得・挿入・削除します。 List の実装のうち、リンクリスト特有のメソッドがいくつか追加されています。 スタックやキューなどのデータ構造を表現したい場合は、LinkedList を使用すると便利です。 addFirst(Object o) / addLast(Object o), getFirst() / getLast(),removeFirst() / removeLast()
Set 系	java.util.Set	Collection を親にもつ重複を許さない集合のインターフェース 特別重要なメソッドは追加されていません
	HashSet	ハッシュを用いた値の重複を許さない順不同の要素集合クラス
	TreeSet	二分探索木を用いた値の重複を許さないキーでソートされた順不同の要素集合クラス
Map 系	java.util.Map	Collection を親にもつキーと値のペアを表す構造のインターフェース、下記の様な抽象メソッド定義がなされています。

		Object put(Object key, Object value), get(Object key), containsKey(Object key), remove(Object key), keySet(), entrySet(), values(), isEmpty(), size()
	HashMap	キーの検索にハッシュを用いたキーと値の組からなる要素の集合クラス 要素は、キーを示して取得・挿入・削除します。
	TreeMap	キーの検索に二分探索木を用いたキーと値の組からなる要素の集合クラス 要素は、キーを示して取得・挿入・削除します。 java.lang.Comparable インターフェースの compareTo(Object o)メソッドを呼び出し、その結果を用いてデータの格納位置を決定しています。

6.2. Collection インタフェースでの iterator 活用

コレクションフレームワークの基底となる java.util.Collection インターフェースには、Iterator（反復子）を取得する java.util.Iterator iterator()メソッドがあります。Iterator にある次のメソッドにより各要素を取得します。

boolean hasNext()：次の要素が存在するか検査する

Object next()：次の要素を取得する

Iterator は、コレクションの種類によらず使用可能です。

```
import java.util.Collection;
import java.util.Iterator;
import java.util.HashSet;

public class App {
  public static void main(String[] args) {
    Collection c = new HashSet();
    c.add("hoge");
    c.add("foo");
    c.add("bar");

    for (Iterator iter = c.iterator(); iter.hasNext();) {
      String element = (String) iter.next();
      System.out.println(element);
    }
  }
}
```

6.3. リスト（List）

6.3.1. ArrayList

ArrayList は配列を扱う一般的なクラスです。

```
import java.util.*;

class ArrayListTest {
  public static void main(String[] args) {
    ArrayList list = new ArrayList();
```

```
        list.add("AAA");
        list.add("BBB");
        list.add("CCC");
        for (int i = 0; i < list.size(); i++) {
            System.out.println(list.get(i));
        }
    }
}
```

下記などのメソッドが用意されています。

list.add(o) : オブジェクト o を配列の末尾に追加する
list.add(n, o) : オブジェクトを n で指定した場所に追加する list.get(n) : n 番目の要素を得る
list.remove(n) : n 番目の要素を削除する
list.set(n, o) : n 番目の要素をオブジェクト o で置き換える
list.size() : 要素の個数を得る
list.isEmpty() : 空かどうか調べる
list.indexOf(o) : オブジェクト o と等しい要素のインデックスを探す
list.contains(o) : オブジェクト o と等しい要素があるか調べる
list.addAll(list2) : 配列の末尾に配列 list2 を追加する

6.3.2. LinkedList

LinkedList も ArrayList と同じように使用することができます。挿入や削除を頻繁に行う場合は ArrayList よりも LinkedList の方が高速です。ただし、get() による参照は ArrayList の方が高速です。

```
import java.util.*;

class LinkedListTest {
    public static void main(String[] args) {
        LinkedList list = new LinkedList();
        list.add("AAA");
        list.add("BBB");
        list.add("CCC");
        for (int i = 0; i < list.size(); i++) {
            System.out.println(list.get(i));
        }
}
```

6.4. セット (Set)

6.4.1. HashSet

HashSet も配列を扱いますが、要素の重複が許されない、順序の保障が無い点が ArrayList や LinkedList と異なります。要素を参照する際には Iterator を用います。

```
import java.util.*;
class App {
    public static void main(String[] args) {
        HashSet set = new HashSet(); set.add("AAA");
        set.add("BBB");
        set.add("CCC");
```

```
        set.add("AAA");
        Iterator it = set.iterator();
        while (it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
```

実行結果は下記のようになります。AAA を 2 回 add() していますが、重複要素がひとつにマージされます。結果の順序は add() した順序に関係なくバラバラになります。

```
user%> java App
CCC AAA BBB
```

下記などのメソッドが用意されています。

set.add(o) : オブジェクト o を配列の末尾に追加する
set.clear() : 配列をクリアする
set.contains(o) : オブジェクト o と等しい要素があるか調べる
set.isEmpty() : 空かどうか調べる
set.remove(o) : オブジェクト o にマッチする要素を削除する
set.size() : 要素の個数を得る

6.4.2. TreeSet

TreeSet も HashSet と同じように使用できます。要素が自動的にソートされる点が HashSet と異なります。

```
import java.util.*;

class App {
    public static void main(String[] args) {
        TreeSet set = new TreeSet();
        set.add("CCC");
        set.add("AAA");
        set.add("BBB");
        set.add("AAA");
        Iterator it = set.iterator();
        while (it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
user%> java App
AAA BBB CCC
```

6.5. マップ（Map）

6.5.1. HashMap

HashMap は、名前（キー）と値の組み合わせを要素として持つ配列です。

```
import java.util.*;
```

```
class App {
   public static void main(String[] args) {
      HashMap map = new HashMap();
      map.put("Name", "Abe");
      map.put("Age", new Integer(26));
      System.out.println("Name = " + map.get("Name"));
      System.out.println("Age = " + map.get("Age"));
   }
}
user%>java App
Name = Abe Age = 26
```

すべての要素を参照するには下記のようにします。

```
Iterator it = map.keySet().iterator();
   while (it.hasNext()) {
   Object o = it.next();
   System.out.println(o + " = " + map.get(o));
}
```

6.5.2. TreeMap

TreeMap も HashMap と同じように使用できます。要素がキーによって自動的にソートされる点が HashMap と異なります。

```
import java.util.*;
class App {
   public static void main(String[] args) {
      TreeMap map = new TreeMap();
      map.put("Name", "Abe");
      map.put("Age", new Integer(26));
      Iterator it = map.keySet().iterator();
      while (it.hasNext()) {
         Object o = it.next();
         System.out.println(o + " = " + map.get(o));
      }
   }
}
```

6.6. コレクションのマルチスレッド対応

コレクションフレームワークで提供されている実装クラスはマルチスレッド環境に対応していません。これは通常のシングルスレッド環境で排他処理などのマルチスレッドのための処理が原因でパフォーマンスが劣化を防ぐためです。

マルチスレッド環境でコレクションを利用する場合には同期化コレクションが必要です。synchronizedList／synchronizedMap／synchronizedSet メソッドを使用して変換して下さい。以下に同期化コレクションに変換した例を示します。

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class App{
  public static void main(String[] args) {
   Listlist = Collections.synchronizedList(new ArrayList<>());
   list.add("Monday");
   list.add("Tuesday");
   list.add("Wednesday");
   System.out.println(list);
 }
}
```

6.7. ジェネリクス

コレクションの要素は全てのオブジェクトの基底クラスである Object でしたが、要素クラスを指定することである要素専門のコレクションにすることが出来ます。そうしたことで、間違った型要素を入れないようにチェックを厳しくし且つ、要素取得時にいちいちキャストで型を指定しなくても良くなります。

コレクションを使用する際は、積極的にジェネリクスを使用して下さい。

例えば次は、List<String>のように<…>の形式で型を明示して String 型専用のリストを作成する例です。

```
List<String> daysOfWeek = new ArrayList<String>();
daysOfWeek.add("Monday");
daysOfWeek.add("Tuesday");
daysOfWeek.add("Wednesday");
String day = daysOfWeek.get(0);
```

以下にメソッドパラメータとして使用する例を示します。

```
static void expurgate(Collection<String> c) {
    for (Iterator<String> i = c.iterator(); i.hasNext(); )
      if (i.next().length() == 4)
        i.remove();
}
```

6.7.1. 境界ワイルドカード型ジェネリクス

ジェネリクスは、型の一致を厳密にチェックできるのが利点ではありますが、様々な数値型クラスのオブジェクトに適用できません。その様な場合、ワイルドカードまたは境界ワイルドカード型を使って型指定を緩やかにすることが出来ます。

```
public static double sumOfList(List<? extends Number> list) {
    double s = 0.0;
    for (Number n : list)
        s += n.doubleValue();
    return s;
}
List li = Arrays.asList(1, 2, 3.5);
System.out.println("sum = " + sumOfList(li));
```

指定された型を上限にその派生クラスを許容する指定を上限境界ワイルドカード型として<? extends E>の形式）で表し、指定された型を下限にそのスーパークラスを許容するものを下限境界ワイルドカード型として<? super E>で表します。

6.7.2. ジェネリクスタイプ

クラスの定義にもジェネリクスは有効に使用できます。例えば、下記のような Parking クラスを作って中にオブジェクトを一つ入れられるとします。オブジェクトの元となるクラスが様々である場合、そのクラスの種類だけ Parking クラスを作るか下記のように汎用的なクラスを作る必要があります。しかしこれでは、型の指定がないのでどんなオブジェクトでも入れられるし取り出したときにもキャストが必要です。

```
public class Parking {
    private HashMap map;

    public void add(Object key, Object value) { this.map.add(key, value); }
    public Object get(Object key) { return this.map.get(key); }
}
```

そのような時に、クラス定義のクラス名にジェネリクスを使用して外部から型を 1 つまたは複数渡してあげることが出来ます。

```
public class Parking<K, V> {
    private HashMap map<K, V>;

    public void add(K key, V value) { this.map.add(key, value); }
    public V get(K key) { return this.map.get(key); }
}

Car carParking = new Parking<String, Car>();
Truck truckParking = new Parking<String, Truck>();
Bus busParking = new Parking<String, Bus>();
```

T にある一定の制限を付けたい場合はどうでしょう？

例えばこの駐車場は、車は使えるがバイクは使えないとするとスーパークラスに automobile クラスが定義されていると次のように書けます。

```
public class Parking<T extends automobile> {
```

型の定義に使うキーワードは次のような暗黙ルールがあるので、可読性のためになるべくルールに従いましょう。

```
E - Element (used extensively by the Java Collections Framework)
K - Key
N - Number
T - Type
V - Value
S,U,V etc. - 2nd, 3rd, 4th types
```

6.8. 変更不可能なリスト、セットおよびマップの作成 [Java9～]

リスト、セットおよびマップの各インタフェースのコンビニエンス静的ファクトリ・メソッドによって、変更不可能なリスト、セットおよびマップを簡単に作成できるようになりました。

```
List<String> stringList = List.of("a", "b", "c");
Set<String> stringSet = Set.of("a", "b", "c");
Map<String, Integer> stringMap = Map.of("a", 1, "b", 2, "c", 3);
```

キーと値のペアの数が 10 を超える場合は、`Map.entry` メソッドを使用してマップのエントリを作成し、それらのオブジェクトを `Map.ofEntries` メソッドに渡します。次に例を示します。

```
import static java.util.Map.entry;
Map <Integer, String> friendMap = Map.ofEntries(
    entry(1, "Tom"),
    entry(2, "Dick"),
    entry(3, "Harry"),
    ...
    entry(99, "Mathilde"));
```

変更可能なコレクションから、変更不可能なコレクションへのコピー

Set.copyOf(set）, Map.copyOf(map)も同様にコピーできます。

```
List<String> list1 = new ArrayList<>();
list1.add("a")
list1.add("b")
list1.add("c")
List<String> snapshot = List.copyOf(list);
```

7. 関数型インタフェースとラムダ式

7. 関数型インタフェースとラムダ式

関数型インターフェースという「抽象メソッドを 1 つだけ持つインターフェース」に短いコードで実装を記述する文法です。ラムダ式は、無名クラスの代替として関数型インターフェースとの組み合わせで真価を発揮し、コードをより簡潔に、直感的に記述できるようになります。

7.1. 無名クラス

無名クラスは名前を持たないクラス定義式です。クラスやインターフェースの定義からインスタンス化までをまとめて 1 文で表現します。ユーザからのイベントリスナーなど、他で使わない場合の使い捨てクラスの実装などに使われます。

```
public interface IColorPrintable {
   String red = "¥u001b[00;31m"; String green = "¥u001b[00;32m";
  String yellow = "¥u001b[00;33m"; String end = "¥u001b[00m";
 void print(String message, String color);
}
public class App {
 public static void main(String[] args) {
  IColorPrintable r = new IColorPrintable() {
   @Override
   public void print(String message, String color) {
    String colorCode = switch(color) {
            case “red” -> red; case “green” -> green;
            case “yellow” -> yellow; default -> end;        };
    System.out.println(colorCode+ message +"color"+end);
   }
  };
  r.print(“Hellow!”, “green”);
 }
}
```

7.2. ラムダ式[Java8～]

ラムダ式は、先ほどの無名クラスなどの記述をもっとシンプルに表現することが出来ます。

ラムダ式で実装できるのは、関数型インタフェースと呼ばれる抽象メソッドを一つしかもたないインターフェースのことです。@FunctionalInterface アノテーションを利用して、明示的に宣言することもできます。

```
@FunctionalInterface
public interface IColorPrintable {
   String red = "¥u001b[00;31m"; String green = "¥u001b[00;32m";
  String yellow = "¥u001b[00;33m"; String end = "¥u001b[00m";
 void print(String message, String color);
}

public class App {
 public static void main(String[] args) {
  IColorPrintable r = (message, color) -> {
    String colorCode = switch(color) {
            case “red” -> red; case “green” -> green;
            case “yellow” -> yellow; default -> end; };
    System.out.println(colorCode + message + "color" + end);
   }
```

```
    };
    r.print("Hellow!", "green");
  }
}
```

ラムダ式では、引数が一つならば"(引数)"の括弧を、またそれぞれの引数の型宣言を型推論により省略できます。さらに処理が return 文だけの場合は return キーワードを省略できます。

```
IAreaOfCircle hoge = (double radius) -> {
  return radius * radius * PI;
};

IAreaOfCircle hoge = radius -> {
  return radius * radius * PI;
};

IAreaOfCircle hoge = radius -> radius * radius * PI;;

IAreaOfCircle hoge = radius -> System.out.println(radius * radius * PI);
```

7.3. メソッド参照

ラムダ式の代わりにメソッド参照という機能を使って、よりメソッドをシンプルな記述が可能な場合があります。

次にラムダ式とメソッド参照の二つを比べるコードを示します

```
// ラムダ式
Function<String, String> toUpperByLambda = s -> s.toUpperCase();
System.out.println(toUpperByLambda.apply("hello")); // HELLO

// メソッド参照
Function<String, String> toUpperByMethodRef = String::toUpperCase;
System.out.println(toUpperByMethodRef.apply("hello")); // HELLO
```

7.4. 関数型インターフェース

ラムダ式で扱える関数型インターフェースは、抽象メソッドを一つしかもたないものであり、static メソッドやデフォルトメソッドや Object クラスにある public メソッドがインターフェース内に抽象メソッドとして定義されているものは無視されます。ラムダ式では関数型インタフェースの定義を使わずに自前のものでも良いですが、様々なインターフェースは既に定義されており、その中でよく使うものを下記に紹介します。

分類	インターフェース	メソッド	説明
処理実行	Consumer<T>	void accept(T t)	T 型引数を受け取り、それを使って処理を行う
	BiConsumer<T,U>	void accept(T t, U u)	T 型と U 型の引数を受け取り、それを使って処理を行う
	例：		

<table>
<tr><td></td><td colspan="3">Consumer<Integer> consumer = num -> System.out.println("num の値: " + num);
System.out.println(consumer.accept(10)); // num の値:10</td></tr>
<tr><td rowspan="4">（処理して）値を返す</td><td>Supplier<T></td><td>T get()</td><td>引数はなく T 型戻り値を返す</td></tr>
<tr><td>Function<T,R></td><td>R apply(T t)</td><td>T 型引数を受け取り演算して R 型戻り値を返す</td></tr>
<tr><td>BiFunction<T,U,R></td><td>R apply(T t, U u)</td><td>T 型と U 型の引数を受け取り演算して R 型戻り値を返す</td></tr>
<tr><td colspan="3">例：
Supplier<String> supplier = () -> "Hello world";
System.out.println(supplier.get()); // Hello world
Function<Integer, String> signedNumber = (i) -> i==0 ? “0” : (i>0 ? “+”+I :”-”+i);
System.out.println(asterisker.apply(10)); // +10</td></tr>
<tr><td rowspan="3">真偽値を返す</td><td>Predicate<T></td><td>boolean test(T t)</td><td>T 型引数を受け取り、戻り値として真偽値を返す</td></tr>
<tr><td>BiPredicate<T,U></td><td>boolean test(T t, U u)</td><td>T 型と U 型の引数を受け取り、戻り値として真偽値を返す</td></tr>
<tr><td colspan="3">例：
BiPredicate<Integer> isLarge = (i, k) -> i > k;
System.out.println(isLarge.test(2, 1)); // true</td></tr>
<tr><td rowspan="3">特定の演算結果を返す</td><td>UnaryOperator<T></td><td>T apply(T t)</td><td>T 型の引数を受け取り、加算、最大値/最小値、String 等の連結などを行い T 型戻り値を返す</td></tr>
<tr><td>BinaryOperator<T></td><td>T apply(T t, T u)</td><td>T 型と U 型の引数を受け取り、加算、最大値/最小値、String 等の連結などを行い T 型戻り値を返す</td></tr>
<tr><td colspan="3">例：
String str = "さん";
UnaryOperator <String> s = x -> x.concat(str);
System.out.println(s.apply("阿部")); //阿部
BinaryOperator<String> concat = (a, b) -> a + b;
System.out.println(concat.apply(“Hello”, ” World”)); // Hello World
BinaryOperator<String> longest = BinaryOperator.maxBy(Comparator.comparingInt(String::length));
System.out.println(longest.apply(“orage”, “pineapple”)); // pineapple
BinaryOperator<String> shortest = BinaryOperator.minBy(Comparator.comparingInt(String::length));
System.out.println(shortest.apply(“orage”, “pineapple”)); // orage</td></tr>
</table>

8. ストリーム API（Stream API）

8. ストリーム API（Stream API）[Java8～]

Stream API は、集合から個々の要素を繰り返し処理のための仕組みです。コレクション、ファイルなどのデータソースから要素を取り出して、これを処理の流れを作成し演算・加工等するしくみを提供します。

ストリームは以下の手順で処理されます。

・データソースから Stream を生成する

・フィルターや加工などの中間処理を行う

・最終的な集計や出力などの終端処理を行う

```
List<String> data = new ArrayList<>(
Arrays.asList("apple", "orange", "  apricot"));
data
    .stream()                          // Stream 生成
    .filter(s -> s.startsWith("a")) // 'a'で始まるものをフィルタリング
    .map(s -> s.toUpperCase()) // 大文字に変換
    .forEach(System.out::println); // APPLE, APRICOT が出力される
```

8.1. ストリーム生成

リスト、文字列など、クラスによっては現在のインスタンスをもとに Stream を生成できます。

対象オブジェクト	メソッド	説明
コレクション	Collection#stream()	List, Set インタフェースコレクションを対象とする Stream
コレクション	Collection#parallelStream()	並列処理を許可された、〃
配列(array)	Arrays.stream(array)	プリミティブな数値配列は、ラッパークラスジェネリクス Stream に変換される
String	String#chars()	一文字づつの文字群 Stream
ファイル	Files.lines(path, charset)	ファイルの各行 Stream
バッファされた各行	BufferedReader#lines()	バッファされた各行を対象とする Stream
乱数生成	Random#ints(size, begin, end)	begin から end（含まれない）までの乱数を 5 回振る

ストリーム生成例を下記に示します。

```
List <Integer> list = List.of(1,2,3);
Stream<List> stream = list.stream();
stream.forEach(System.out::println);

Map<String, String> map = Map.of("apple", “read”, "orange", “yellow”, "grape", “green”);
Stream<Set> stream = map.entrySet().stream();
stream.forEach(System.out::println);


String[] array = {"apple", "orange", "apricot"};
Stream<String> stream = Arrays.stream(array);
stream.forEach(System.out::println);
```

```
int[] array = {1,2,3};
IntStream stream = Arrays.stream(array);
stream.forEach(System.out::println);

Double[] array = {0.1, 1.2, 2.3};
Stream<Double> stream = Arrays.stream(array);
stream.forEach(System.out::println);
```

Stream クラスには、幾つかの有用なメソッドがあります

static <T> Stream<T> of(T... values)

指定された値を要素に持つ、順序付けされた順次ストリームを返します。

```
Stream<String> stream = Stream.of("apple", "orange", "apricot")
stream.forEach(System.out::println);
```

static <T> Stream<T> generate(Supplier<T> s)

指定された Supplier によって生成される要素を含む、順序付けされていない無限順次ストリームを返します

```
Stream<Double> stream = Stream.generate(() -> Math.random());
stream.limit(3).forEach(System.out::println);
```

static <T> Stream<T> iterate(T seed, UnaryOperator<T> f)

初期要素 seed に関数 f を繰り返し適用することで生成される、順序付けされた無限順次 Stream を返します(seed、f(seed)、f(f(seed))、といった要素から成る Stream が生成される)。

```
Stream<Integer> stream = Stream.iterate(2, x -> x <= 256, x -> x * 2);
stream.forEach(System.out::println); //2481632128256
```

static <T> Stream<T> concat(Stream<? extends T> a, Stream<? extends T> b)

最初のストリームの全要素と 2 番目のストリームの全要素を連結したものを要素に持つ、遅延連結ストリームを作成します。 結果のストリームが順序付けされるのは、入力ストリームがどちらも順序付けされている場合であり、並列的になるのは、入力ストリームのどちらかが並列的な場合です。

```
Stream<String> stream1 = Stream.of("apple", "orange", "apricot");
Stream<String> stream2 = Stream.of("strawberries", "grape", “lemon");
Stream<String> stream = Stream.concat(stream1, stream2)
stream.forEach(System.out::println);
```

8.2. ストリーム中間処理

何らかのフィルターや加工などの中間処理を行います、複数の中間処理を任意の順番で重ね合わせ可能です。

よく使用する中間処理メソッドを下記に示します。

IntStream filter(IntPredicate predicate)

要素に絞り込みます

```
List <Integer> list = List.of(1,2,3,4,5,6,7);
list.stream()
    .filter(x -> x<3)
    .forEach(System.out::print);
```

IntStream distinct()

重複を取り除きます

```
List <Integer> list = List.of(1,2,2,3,4,5,5);
list.stream()
    .distinct()
    .forEach(System.out::print);
```

IntStream limit(long maxSize)

数を制限します

```
List <Integer> list = List.of(1,2,3,4,5);
list.stream()
    .limit(3)
    .forEach(System.out::print);
```

IntStream skip(long n)

引数で指定した数以降の要素を取り出す

```
List <Integer> list = List.of(1,2,3,4,5);
list.stream()
    .skip(3)
    .forEach(System.out::print);
```

IntStream sorted()

ストリーム内の要素を並び替えます

```
List <String> list = List.of("G","E","A","B","C");
list.stream()
    .sorted()
    .forEach(System.out::print);
```

IntStream map(IntUnaryOperator mapper)

ストリームの要素に対して何らかの処理を加えます

```
List <String> list = List.of("apple","banana","orange");
list.stream()
    .map(x -> x.substring(3))
    .forEach(System.out::print);
```

IntStream peek(IntConsumer action)

終端操作までの間にどのような状態になっているか確認し、要素がパイプライン内の特定のポイントを通過する際にその内容を確認するようなデバッグをサポートする

```
List<Integer> list = Arrays.asList(7,2,3,10,5);
list.stream()
  .filter(x -> x % 2 == 0) //偶数のみ抽出
  .peek(x -> {
    System.out.println("debug:" + x);
  })
  .sorted()
  .forEach(x->{
    System.out.println("value:" +x);
  });
```

8.3. ストリーム終端処理

終端処理では、Stream 内で最終的な集計や出力などを行います。Stream は中間処理の時点では処理をストックしいて終端処理が呼び出されたところでまとめて中間処理も含めて終端処理を実施します。終端処理は末尾で必ず呼び出さなければなりません。

8.3.1. collect メソッド

collect()メソッドは、ストリームの要素を集計・編集し、List、Set、Map などのさまざまなデータ構造に変換する汎用的な終端操作です。通常、Collectors クラスの静的メソッドで提供されるコレクタを引数に指定して使用します。

static <T> Collector<T,?,List<T>> Collectors.toList(): ストリームの要素を List に収集します。

static <T> Collector<T,?,Set<T>> Collectors.toSet(): ストリームの要素を Set に収集します。

tatic <T,K,U> Collector<T,?,Map<K,U>> Collectors.toMap(keyMapper, valueMapper): ストリームの要素を Map に収集します。キーと値のマッピング関数を指定する。

Collectors.joining(delimiter): 文字列のストリームを、指定した区切り文字で連結した String を生成します。

```
import java.util.List;
import java.util.stream.Collectors;
import java.util.Arrays;

public class App {
  public static void main(String[] args) {
    List<String> fruits = Arrays.asList("apple", "banana", "orange", "apple");

    // 1. List に収集する例
    List<String> list = fruits.stream().filter(f -> f.length() > 5) // 5 文字より長いフルーツをフィルタリング
                        .collect(Collectors.toList()); // リストに収集
    System.out.println("Filtered List: " + list); // 出力例: Filtered List: [banana, orange]

    // 2. 文字列を連結する例
    String str = fruits.stream().collect(Collectors.joining(", ")); // カンマ区切りで連結
```

```
        System.out.println("Joined Fruits: " + str); // 出力例: Joined Fruits: apple, banana, orange, apple
    }
}
```

8.3.2. reduce メソッド

int reduce(int identity, IntBinaryOperator op)

値を累積的に結合していく関数を実行するためのものです。

```
List<Integer> list = List.of(1, 2, 3);
Optional<Integer>result=list.stream().reduce((a,b) -> a + b);
result.ifPresent(System.out::print); // 6
```

reduce を使った終端処理の例を下記に示します。

```
import java.util.List;

record Player (String name, int experience) {}

/**
 * Stream API の reduce 操作
 * ストリームの要素を一つにまとめる強力な終端処理を学びます。
 */
public class App {
    public static void main(String[] args) {
        System.out.println("--- Stream API の reduce 操作 ---");

        List<Player_1201> party = List.of(
            new Player_1201("勇者", 1500),
            new Player_1201("戦士", 1200),
            new Player_1201("魔法使い", 1800)
        );

        // --- reduce で合計値を計算 ---
        // `reduce`は、ストリームの要素を一つずつ取り出し、演算を繰り返して一つの結果にまとめます。
        // ビデオシナリオ解説：
        // (0, 1500) -> 1500
        // (1500, 1200) -> 2700
        // (2700, 1800) -> 4500　という流れです。

        // `reduce(初期値, (累積値, 要素) -> 新しい累積値)`
        int totalExperience = party.stream()
            .map(player -> player.experience()) // プレイヤーの経験値(Integer)のストリームに変換
            .reduce(0, (subtotal, experience) -> subtotal + experience);

        System.out.println("パーティの総経験値: " + totalExperience);

        // メソッド参照を使うとさらに簡潔に書けます
        int totalExperience2 = party.stream()
            .mapToInt(player -> player.experience()) // IntStream を使うとより効率的
```

```
            .sum(); // sum()は reduce の一種です

        System.out.println("パーティの総経験値 (sum()を使用): " + totalExperience2);

        // --- reduce で最も名前が長いプレイヤーを探す ---
        party.stream()
            .reduce((p1, p2) -> p1.name().length() > p2.name().length() ? p1 : p2)
            .ifPresent(p -> System.out.println("最も名前が長いプレイヤー: " + p.name()));
    }
}
```

8.3.3. その他の終端処理メソッド

void forEach(IntConsumer action)

ストリーム内の要素をコレクションが保持している順に処理する

```
List <String> list = List.of("apple","banana","orange");
list.stream().forEach(System.out::print);
```

OptionalInt findFirst()

ストリーム内の最初の要素を取り出すメソッドです。

```
List <String> list = List.of("apple","banana","orange");
Stream<String> stream = Arrays.stream(array);
stream.findFirst().ifPresent(System.out::println);
```

OptionalInt max()

最大値を戻します。

```
List<Integer> list = List.of(1,2,3,4,5);
Optional <Integer> result = list.stream().max((x,y)->{
            return x==y ? 0: (x>y ? 1:-1);
   });
result.ifPresent(System.out::println);
```

OptionalInt min()

最小値を戻します。

```
List<Integer> list = List.of(1,2,3,4,5);
Optional <Integer> result = list.stream().min((x,y)->{
      return x==y ? 0: (x>y ? 1:-1);
   });
result.ifPresent(System.out::println);
```

９．例外処理

9. 例外処理

9.1. 例外とは

Java では、プログラムを実行する最中に発生するエラーを 例外（exception）として扱うことができます。例外には例えば、0 で割り算をした、メモリが枯渇した、存在しないファイルを開こうとしたなどがあります。

9.2. 例外をキャッチする（try, catch, finally）

FileReader() は、 指定したファイルが存在しない場合に FileNotFoundException 例 外 を 発 生 します 。また、readLine ()は、IOException を発生します。

try は、例外を発生させる可能性のある処理を呼び出す場合に用います。try { ... } の間で例外が発生した場合、catch を用いてこの例外を捕捉します。下記の例では、例外が発生した場合に、その内容を標準出力に書き出します。このような例外発生時の処理を 例外ハンドラ と呼びます。

```
try {
   FileReader in = new FileReader("data.txt");
   BufferedReader br = new BufferedReader(in);
   System.out.println(br.readLine());
} catch (Exception e) {
   System.out.println(e);
}
```

catch は例外発生時に投げられる 例外オブジェクト に応じて複数記述することができます。いずれの catch にも該当しない例外やファイルクローズなどの後処理を行うために、finally を用います。 finally の部分は、たとえ try 部分で return を行っても実行されます。finally は、try で使用するリソースの解放（後始末）をする記述に使用しますが、一般的なリソースの取得と解放には、後で述べる try-with-resources 文を使用することをお勧めします。

```
try {
...
} catch (TestAException e) {
...
} catch (TestBException e) {
...
} finally {
...
}
```

例外が発生した場合、try { ... } catch の間の残りの処理はスキップされます。

下記の例では、正常終了の場合(1) (2) (3) (4) (5) (8) (9) (10)の処理が実行されます。FileReader("data.txt")で FileNotFoundException 例外が発生した場合 (1) (2) (6) (9) (10) の処理が実行され、br.readLine()で IOException 例外が発生した場合は(1) (2) (3) (4) (7) (8) (9) (10)の処理が実行されます。

FileNotFoundException は、IOException のサブクラス更に IOExcepiton は Exception のサブクラスなので、catch (FileNotFoundException e)と catch (IOException e)を纏めて、catch (IOException e)や catch (Exception e)と 1 行で書いても良いです。

```
System.out.println("(1)");
BufferedReader br = null;
try {
    System.out.println("(2)");
    FileReader in = new FileReader("data.txt");        //← FileNotFoundException 例外発生?
    System.out.println("(3)");
    br = new BufferedReader(in);
    System.out.println("(4)");
    System.out.println(br.readLine());                 //← IOException 例外発生?
    System.out.println("(5)");
} catch (FileNotFoundException e){
    System.out.println("(6)");
} catch (IOException e){
    System.out.println("(7)");
} finally {
    if (in != null) {
        br.close();
        System.out.println("(8)");
    }
    System.out.println("(9)");
}
System.out.println("(10)");
```

9.3. try-with-resources 文 [Java7～, 9～]

例外時のリソースの解放忘れや、厳密に言うと finally 句の close メソッド呼出で IOException 等のリスクを減らすため finaly ではなく try-with-resources の利用できます。try 文の括弧内には、AutoCloseable インターフェースを実装したオブジェクトの複数のリソースを宣言可能で、ここで定義されたものは正常時・例外時に関わらず正しい順番に必要な分だけ自動的にリリースされます。

```
try (FileReader in = new FileReader("data.txt");
    BufferedReader br = new BufferedReader(in);){
    System.out.println(br.readLine());
} catch (IOException e) {
    System.out.println(e);
}
```

また、Java9 からは try 節でのクラス宣言の不要になりました。

```
FileReader in = new FileReader("data.txt");
BufferedReader br = new BufferedReader(in);
try (in; br){
    System.out.println(br.readLine());
} catch (IOException e) {
    System.out.println(e);
}
```

9.4.　例外を投げる（throw, throws）

throw を用いて例外を投げることができます。例外を投げるメソッドを定義する場合は、throws を用いて、そのメソッドが例外を投げる可能性があることを明示しておきます。throws　宣言されたメソッドは、try catch　で例外を補足しないと、コンパイル時にエラーとなります。

```
class MyException extends Exception {
   public String errMsg;
      MyException(String msg) {
      errMsg = msg;
   }
}

class ThrowTest {
   public static void main(String[] args) {
   try {
      methodA();
   } catch (MyException e) {
      System.out.println(e.errMsg);
   }
}

static void methodA() throws MyException {
   MyException e = new MyException("エラー発生!!");
   throw e;
   }
}
```

9.5.　例外オブジェクト（Exception）

例外オブジェクトは次のようなクラス階層を持ちます。

□Object

└□Throwable

├□Error

│ ├□

└□Exception

├□RuntimeException

│ ├□

├□

├□

それぞれのクラス（およびそのサブクラス）は次のように分類できます。

基底クラス	例外クラス	説明
Error	OutOfMemoryError	ヒープ領域が不足して JVM がオブジェクトを生成できない
	VirtualMachineError	JVM が壊れている
	StackOverflowError	スタック領域が不足している
Exception	IOException	入出力関連例外
	SQLException	データベース関連例外
	PrinterException	印刷関連例外
RuntimeException	AritimetricException	0 による除算など
	ArrayIndexOutOfBoundsException	配列のインデックスが範囲外
	ClassCastException	不当なキャストを行った
	IllegalArgumentException	引数不正
	NumberFomatException	数値フォーマットが異なる文字列を数値に変換しようとしている
	NullPointerException	変数の値が null 値で参照しようとした

Error クラスは、JavaVM で検出される致命的エラーで、例外ハンドラを記述しても処理を継続できない様な致命的なものですプログラムやシステムの実行環境を見直した方が良いでしょう。

RuntimeExcption クラスを継承した例外は非チェック例外とよばれ、バグがあるからで通常アプリケーションで例外処理しせず、これが起こった場合は原因となるバグを修正すべきものです。

RuntimeException 以外の Exception は、プログラム動作中に起こり得るものなのでエラーハンドリングが必要です。

9.6. Optional クラス［Java8～］

java.util.Optional クラスは、処理中に Null が発生する可能性があるか明らかにし NullPointerException のハンドリングを省略し簡素化することの出来る機能です。

今まで、Null の可能性のある値を扱う処理には if 文で値が Null でない場合というチェックを入れたり、try～catch で例外をハンドリングしていましたが、Optional のメソッドを使って効率的に事前処理が可能になりました。

次の様に従業員クラスがあって、従業員 ID から名前を取得して表示する場合を考えてみます。

getNameById()メソッドからは見つからない場合は、null が返るのでそのための処理をしないとなりません。

```
import java.util.Map;

public class EmployeeMap {
   static public string TITLE = “さん”;
   // key:employee ID, value:employee name
   private Map<String, String> map;
   public EmployeeMap(Map<String, String> map) {
      this.map = map;
   }

   // return name of employee id
   public String getNameById(String id) {
      return this.map.get(id);
   }
}

EmployeeMap employeeMap = new EmployeeMap(Map.of(
   “2025_04_01-001”, “Jhone”,
   “2025_04_01-002”, “Meary”,
   “2025_05_01-001”, “Steave”,
   );
String name = employeeMap.getNameById(“2022_04_01-002”);
if (name == null) {
   System.out.println(“not find”);
}else{
   System.out.println(name);
}
```

上記の EmployeeMap#getNameById()メソッドを Optional を使って書き換えて、それを使ってみましょう

メソッドの戻り値を Optional<戻り値の型>として、Optional.ofNullable()は null である可能性のある値とします。

```
import java.util.Map;
import java.util.Optional;

public class EmployeeMap {
   static public string TITLE = “さん”;
   // key:employee ID, value:employee name
   private Map<String, String> map;
   public EmployeeMap(Map<String, String> map) {
      this.map = map;
```

```
    }

    // return name of employee id
    public Optional<String> getNameById(String id) {
        return Optional.ofNullable(this.map.get(id));
    }
}

EmployeeMap employeeMap = new EmployeeMap(Map.of(
    “2025_04_01-001”, “Jone”,
    “2025_04_01-002”, “Meary”,
    “2025_05_01-001”, “Steave”,
    );
```

Optional にある各種メソッドの適用例を下記に示します。

```
Optional<String> opt = employeeMap.getNameById(“2022_04_01-002”);

// Optional.get()は Optional から値を取得します。
// この場合 null なので NoSuchElementException が throw されます。
System.out.println(opt.get());
// orElse は null の場合は代替え値を指定します
System.out.println(opt.orElse("No name").concat(EmployeeMap.TITLE));
// ifPresent は、値がある場合だけ処理します
opt.ifPresent(v -> System.out.println(v.concat(EmployeeMap.TITLE)));
// ifPresentOrElse は、値があるまたはない場合それぞれの処理を書くことができます
opt.ifPresentOrElse(v -> System.out.println(v.concat(EmployeeMap.TITLE)), ()->System.out.println("No name"));
// map は値がある時だけ処理します
opt.map(v -> v.concat(EmployeeMap.TITLE)).ifPresent(v -> System.out.println(v));
System.out.println(opt.map(v -> v.concat(EmployeeMap.TITLE)).orElse("No name"))
// filter は括弧内の条件を満たすものを選別します
opt.filter(v -> v.contains(“Meary”)).ifPresent(v -> System.out.println(v));;
// flatMap は Optional が入れ子にならないように、値が存在する場合に変換を適用し結果の Optional をフラット化します
Optional<String> opt2 = opt.flatMap(v -> " Meary".equals(v) ? Optional.of(“Miss ”+v) : Optional.empty());

// Optional が持っている値から直接 Stream を作れます
Optional<List<String>> opt = Optional.of(Arrays.asList("apple", "orange"));
opt.stream().flatMap(v -> v.stream()).forEach(v -> System.out.println(v)); // "apple"、"orange"が出力される

Optional<List<String>> opt2 = Optional.empty();
opt2.stream().flatMap(v -> v.stream()).forEach(v -> System.out.println(v)); // エラーにならず何も出力されない
```

10. 入出力(File-I/O)

10.入出力(File-I/O)

データの 入出力 には多数のクラスが用意されています。File クラスはファイルやパスを扱うことが出来ます。また InputStream 系クラスは主にバイナリデータを、また Reader 系クラスは主にテキストデータを扱う際に用いられます。

10.1. ファイル操作

java.io.FIle クラスは、ファイルまたはパス名を抽象的に扱いファイルやファイル属性を操作するメソッドを提供します。

File(File parent, String child)：親抽象パス名および子パス名文字列から新しい File のインスタンスを生成
File(String pathname)：指定されたパス名文字列を抽象パス名に変換して、新しい File のインスタンスを生成
File(String parent, String child)：親パス名文字列および子パス名文字列から新しい File のインスタンスを生成
File(URI uri)：指定された file: URI を抽象パス名に変換して、新しい File のインスタンスを生成

boolean createNewFile()：この抽象パス名が示す空の新しいファイルを生成
static File createTempFile(String prefix, String suffix)：指定された接頭辞と接尾辞をファイル名の生成に使用して、デフォルトの一時ファイル・ディレクトリに空のファイルを生成
boolean delete()：この抽象パス名が示すファイルまたはディレクトリを削除
boolean equals(Object obj) ：この抽象パス名が指定されたオブジェクトと等しいかどうかを判定
boolean exists()：この抽象パス名が示すファイルまたはディレクトリが存在するかどうかを判定
File getAbsoluteFile()：この抽象パス名の絶対形式を返す。
String getAbsolutePath()：この抽象パス名の絶対パス名文字列を返す。
String getName() ：この抽象パス名が示すファイルまたはディレクトリの名前を返す。
String getParent()：この抽象パス名の親のパス名文字列を返す。
String getPath()：この抽象パス名をパス名文字列に変換
boolean isDirectory()：この抽象パス名が示すファイルがディレクトリであるかどうかを判定
boolean isFile()：この抽象パス名が示すファイルが普通のファイルかどうかを判定
long length()：この抽象パス名に指定されているファイルの長さを返す。
String[] list(FilenameFilter filter)：この抽象パス名が示すディレクトリにあるファイルおよびディレクトリの中で、指定されたフィルタの基準を満たすものの文字列の配列を返す。
File[] listFiles(FileFilter filter)：この抽象パス名が示すディレクトリにあるファイルおよびディレクトリの中で、指定されたフィルタの基準を満たすものの抽象パス名の配列を返す。
boolean mkdir()：この抽象パス名が示すディレクトリを生成
boolean mkdirs()：この抽象パス名が示すディレクトリを存在していないが必要な親ディレクトリも作成
boolean renameTo(File dest)：この抽象パス名が示すファイルの名前を変更します。
boolean setReadOnly()：この抽象パス名が示すファイルまたはディレクトリに読込みオペレーションだけが許可
boolean setWritable(boolean writable)：この抽象パス名に所有者の書込み権を設定
Path toPath()：この抽象パスから構築された java.nio.file.Path オブジェクトを返す。
String toString()：この抽象パス名のパス名文字列を返す。
URI toURI()：この抽象パス名を表す file: URI を構築

以下に例を示します。

```
import java.io.File;

public class App{
  public static void main(String[] args) {
    Path path = Paths.get("subdir", "sample.txt");
      // ←Paths.get は複数サブディレクトリを連結してパスを作ります
    try () {
      File file = new File(path);
      System.out.println("Name: "+ file.getName());          // ファイルの名前取得
      System.out.println("Path: "+ file.getPath());           // ファイルのパス取得
      System.out.println("Path: "+ file.getAbsolutePath());   // ファイルの絶対パス取得
```

```
        if (!file.exists()) {                          // ファイルが存在チェック
            file.createNewFile();                      // ファイルの作成
        }
        file.delete();                                 // ファイルを削除
    } catch (IOException e) {
        System.out.println(e);
    }
  }
}
```

Files.lines を使えば、Java 8 以降でファイルの各行を持つ Stream を作れます。ファイルの内容を入力元とした Stream を簡単に作れるので便利ですが巨大なファイルでは OutOfMemoryError が起こるので使わないようにしましょう。

```
// Java 8 以降
Path path = Paths.get("sample.txt");
Files.lines(path).forEach(System.out::println);
```

10.2. バイナリファイルを読み書きする（FileInputStream／FileOutputStream）

FileInputStream／FileOutputStream は指定したファイルをバイナリファイルとして読み書きします。

・FileInputStream - バイナリファイルを読み込む
public FileInputStream(String name) throws FileNotFoundException
：パス名 name で指定される FileInputStream を作成します。
public FileInputStream(File file) throws FileNotFoundException
： File オブジェクト file で指定される FileInputStream を作成します。
void close()：ストリームを閉じ、関連するシステム・リソースを解放します。
int read()：データのバイトを読み込みます。
int read(byte[] b)：最大 b.length バイトのデータを、バイト配列に読み込みます。
int read(byte[] b, int off, int len)：最大 len バイトのデータを、バイト配列に読み込みます。

・FileOutputStream - バイナリファイルを書き込む
public FileOutputStream(String name[,boolean append]) throws FileNotFoundException
：パス名 name で指定される FileOutputStream を作成します。
public FileOutputStream(File file[,boolean append]) throws FileNotFoundException
： File オブジェクト file で指定される FileOutputStream を作成します。
name：ファイルのパス append：追記モードで開くか
void close()：ストリームを閉じ、関連するシステム・リソースを解放します。
void flush()：ストリームをフラッシュします。
void write(int b)：指定されたバイトをこのファイル出力ストリームに書き込みます。
void write(byte[] b)：指定されたバイト配列の b.length バイトをファイル出力ストリームに書き込みます。
void write(byte[] b, int off, int len)：指定されたバイト配列のオフセット位置 off から始まる len バイトをこのファイル出力ストリームに書き込みます。

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
class App {
  public static void main(String[] args) {
    try(FileInputStream in = new FileInputStream("file.txt");
        FileOutputStream out = new FileOutputStream("file2.txt");) {
        byte buffer[] = new byte[256];
```

```
        while ((len = in.read(buffer)) != -1) {
            out.write(buffer, 0, len);
            for (int i=0; i<len; i++) {
                System.out.print(Integer.toHexString(buffer[i]) + " ");
            }
        }
        // out.flush(); close() メソッドは内部的に flush() の処理も含むため必要ない
    } catch (IOException e) {
        System.out.println(e);
    }
  }
}
```

10.3. テキストファイルを読み書きする（InputStreamReader／OutputStreamWriter）

InputStreamReader／OutputStreamWriter は指定したストリームを、指定した文字コードで構成されるテキストファイルとして読み下記します。

・InputStreamReader – バイト・ストリームから文字ストリームへの橋渡し
public InputStreamReader(InputStream in, String charset) throws throws UnsupportedEncodingException
：バイト・ストリームから指定された charset でデコードして文字ストリームへの橋渡し
void close()：ストリームを閉じ、関連するシステム・リソースを解放します。
int read()：データのバイトを読み込みます。
int read()：単一の文字を読み込みます。
int read(char[] cbuf, int offset, int length)：配列の一部に文字を読み込みます。
・OutputStreamWriter - 文字ストリームからバイト・ストリームへの橋渡し
public OutputStreamWriter(OutputStream out, String charset) throws UnsupportedEncodingException
：文字ストリームから指定された charset でエンコードしてバイト・ストリームへ渡します。
void close()：ストリームを閉じ、関連するシステム・リソースを解放します。
void flush()ストリームをフラッシュします。
void write(int c)：単一の文字を書き込みます。
void write(char[] cbuf, int off, int len)：文字の配列の一部を書き込みます。
void write(String str, int off, int len)：文字列の一部を書き込みます。

下記は、ファイルのバイトストリームから文字ストリームへデータを読み込み、そしてバイトストリームへファイルを書き込んでいます。「BufferedReader／BufferedWriter」は他の入出力ストリームを利用する「フィルタ系」の、文字単位入出力クラスです。これら 2 つのクラスは入出力データをバッファリングする事により、効率的な入出力 1 行単位でファイルから文字列を読み込むに役立ちます。

```
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.IOException;

class App {
  public static void main(String[] args) {
    try( BufferedReader reader = new BufferedReader(
          new InputStreamReader(new FileInputStream("utf-8.txt"),"UTF-8"));
      BufferedWriter writer = new BufferedWriter(
          new OutputStreamWriter(new FileOutputStream ("sjis.txt"),"Shift_JIS"));
    ){
      String line;
      while((line=reader.readLine())!=null){
```

```
            writer.write(line);
            writer.newLine();
                System.out.print(line);
            }
            // write.flush(); close() メソッドは内部的に flush() の処理も含むため必要ない
        } catch (IOException e) {
                System.out.println(e);
        }
    }
}
```

JDK 17 以前 デフォルトの文字コード OS や実行環境に依存します。

Windows: 日本語環境では Shift_JIS（MS932）になることが多く、海外の環境では別の文字コードになることがあります。macOS や多くの Linux: UTF-8 になることが多いです。

このため、特定の環境で作成・実行したプログラムを別の環境で実行すると、デフォルトの文字コードの違いから文字化けが発生する可能性が高くなります。

JDK 18 以降デフォルトの文字コードは UTF-8 です。これにより、どのプラットフォームで実行しても、文字コードを指定しない限り UTF-8 で処理されるため、互換性の問題が軽減されました。

10.4. File I/O で Stream を扱う手順

BufferdReader クラスの lines メソッドを使って Stream を取得し、様々な Stream API 処理を追加することが出来ます。

Stream<String> lines()：Stream を返します。要素は BufferedReader から読み込まれる行です。

```
import java.io.FileInputStream;
import java.io.InputStreamReader;
import java.io.BufferedReader;
import java.io.IOException;
import java.util.stream.Stream;

public class FileStreamExample {
    public static void main(String[] args) {
        String filePath = "sample.txt"; // 読み込むファイルのパスを指定

        try (
            FileInputStream fis = new FileInputStream(filePath);
            InputStreamReader isr = new InputStreamReader(fis, "UTF-8"); // 文字エンコーディングを指定
            BufferedReader br = new BufferedReader(isr)
        ) {
            Stream<String> lines = br.lines();
            lines.forEach(System.out::println); // ストリームを処理する例
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

１１．アノテーション

11. アノテーション

アノテーションとは、“注釈”や“注記”を意味し、クラス・メンバー・引数などに対して付与できるメタ情報です。アノテーションはロジックそのものを変えるものではありませんが、コードにルールを適用し意図・用途を定義することなどが出来ます。アノテーションは、“@”から始まるキーワードで表現され、具体的には大きく以下の 3 つの用途で用いられます。

1. コードにルールを定義してミスをなくす
2. コンパイラの警告メッセージを出なくする
3. プログラムの実行環境に合わせて動作を変える

11.1. 標準アノテーション

標準アノテーションとは、JavaSE で提供されているアノテーションです。よく使われるものを下記に示します。

アノテーション	適用対象	概要
@Override	METHOD	スーパークラスのメソッドをオーバーライドしていると宣言する
@Deprecated	TYPE、METHOD、FIELD、CONSTRUCTOR、LOCAL_VARIABLE、PARAMETER など	クラスやメソッドが「非推奨」であることを宣言する
@SuppressWarnings	TYPE、METHOD、FIELD、CONSTRUCTOR、LOCAL_VARIABLE、PARAMETER など	コンパイラの警告を抑制する
@SafeVarargs	CONSTRUCTOR、METHOD	可変長引数のデータ型の安全性を保障する
@FunctionalInterface	TYPE	関数型インターフェース定義を宣言する

@SuppressWarnings (XXXX)のカッコの中には、次の対象とする警告名を指定します。

警告名	意味
cast	不要なキャスト
deprecation	非推奨のメンバーを使用
finally	finally ブロックの処理が正常に実行されない
null	NULL ポインターエラー
unchecked	危険なキャストを行っている
divzero	０除算
unused	未使用の変数がある

11.2. アノテーションのカスタマイズ

次の@interface を利用して、独自のアノテーションを作ることが出来ます。

```
@interface アノテーション名 {
    型 フィールド名() default 値;
    ........
}
```

例を示します

```
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.METHOD, ElementType.FIELD, ElementType.PARAMETER})
@Documented
public @interface TargetFunction {
  String value();
   double version();
}
```

アノテーション定義では、アノテーションそのものの情報を定義するためのメタアノテーションを付与します。

以下に主なメタアノテーションをまとめます。

メタアノテーション	
@Retention	アノテーションの保持ポリシー（SOURCE、CLASS、RUNTIME） 省略されると、「保持ポリシー」は CLASS SOURCE … コンパイラによって評価され、コンパイル後は破棄される。 CLASS … コンパイル後にクラスファイルに記録される、実行時には廃棄される。 RUNTIME … 実行時まで保持される。クラスやメンバーにアクセスするためのリフレクション API でアノテーションの情報を取得可能。
@Target	アノテーションの付与対象（TYPE、METHOD、FIELD、PARAMETER など） 省略されると、「付与対象」はすべての要素
@inherited	アノテーション情報がサブクラスにも継承されるか？
@Documented	当該アノテーションがドキュメント対象か？ （Javadoc コマンドによる「API リファレンス」など）

作成したアノテーションの使用例を示します。

```
@TargetFunction("挨拶機能")
public class Greeting {
    private String name;
    public Greeting(String name) {
        this.name = name;
    }
    @TargetFunction(value="挨拶機能", version="1.2")
    public void sayHello(String targetName) {
        System.out.println("Hello! Mr. " + targetName + ", My name is" + this.name + ".");
    }
```

```
}
```

リフレクション API を使って、アノテーション情報を取得して処理を行う例を示します。

```
import java.lang.reflect.Method;

public class App {

  public static void main(String[] args) {
    try {
      Method m = Greeting.class.getMethod("sayHello");
      TargetFunction target = m.getAnnotation(TargetFunction.class);
      if (target.value().equals("挨拶機能")) && target.version() >= 1.0) {
        Greeting g = new Greeting("Jone");
         g.sayHello("alice");
      }
    } catch (Exception e) {
      e.printStackTrace();
    }
  }
}
```